

GraphHunter

A CLRS-Style Architecture Reference

Lucas Sotomayor
Base4Security

April 17, 2026

Contents

Preface	vi
I Preliminaries	1
1 Introduction	2
1.1 Problem domain	2
1.2 System overview	2
1.3 What this reference covers	3
2 Notation and Preliminaries	4
2.1 Graphs	4
2.2 Paths and hypotheses	4
2.3 Asymptotic notation	5
2.4 Memory model	5
2.5 Source-pointer convention	5
II Core Data Structures	6
3 The CompactRelation Encoding	7
3.1 Layout	7
3.2 MVCC encoding	8
3.3 Why not <code>#[repr(C, packed)]</code> ?	8
4 The String Interner	9
4.1 Structure	9
4.2 Algorithms	9
4.3 Metrics	10
4.4 The trait seam	10
5 The In-Memory Graph	12
5.1 Fields	12
5.2 Insertion algorithms	12
5.3 Finalization	13
6 The Spillable Edge Store	14
6.1 Modes	14
6.2 Append	14
6.3 Finalization via k -way merge	15
6.4 Reads	16

7 The Metadata Store	17
7.1 Structure	17
7.2 Operations	17
7.3 Immutability	18
8 The Hypothesis AST	19
8.1 Types	19
8.2 Invariants	20
8.3 Static compatibility check	20
III Core Algorithms	21
9 DSL Parsing	22
9.1 Grammar	22
9.2 Algorithm	22
10 Temporal Pattern Matching	24
10.1 Shared state	24
10.2 Entry point: SEARCH-TEMPORAL-PATTERN	25
10.3 The iterative DFS	25
10.4 The smart variant	27
10.5 Parallelism	27
11 SIMD Acceleration	29
11.1 libgraphmatch FFI bridge	29
11.2 Pure-Rust AVX2 intersection	29
11.3 Roadmap	30
12 Anomaly Scoring	33
12.1 State and finalisation	33
12.2 Neighbourhood concentration	34
12.3 Scoring a path	34
12.4 The trait seam (P2-A)	34
12.5 Node-level fast estimate	35
13 Centrality and Analytics	37
13.1 Betweenness centrality (sampled Brandes)	37
13.2 Temporal PageRank	39
13.3 Relation schema	39
14 Path Deduplication and Pagination	41
14.1 Dedup modes	41
14.2 Algorithm	41
14.3 Sort stability	43
14.4 Pagination contract	43
15 Hunt Diff	44
15.1 Classical event-time variant	44
15.2 Semantic gotcha: late-arriving data	44
15.3 The snapshot variant (deferred)	45
15.4 Input validation	45

IV	Ingestion Pipeline	46
16	The LogSource Trait	47
16.1	Structure	47
16.2	Implementations	47
17	Parsers	49
17.1	Trait	49
17.2	Implementations	49
17.3	Parser dispatch	50
18	Writer and Backpressure	51
18.1	Pipeline shape	51
18.2	Modules	51
18.3	Lock contention	51
19	MVCC Edge-Level Stamping	53
19.1	Current state	53
19.2	Follow-up work	53
20	Overflow Persistence	54
20.1	Invariants	54
20.2	File format	54
V	Extension Seams	55
21	GraphRead / GraphWrite Traits	56
21.1	Traits	56
21.2	What's on the trait vs off	56
21.3	Dual-impl contract	57
22	Scorer and ScoreComponent Traits	58
22.1	Traits	58
22.2	Composite orchestrator	58
22.3	Migration	59
23	StringInternerBackend Trait	60
23.1	Contract	60
23.2	Trigger conditions	60
24	IngestAdapter Trait	61
24.1	Contract	61
24.2	Generic implementation	61
24.3	MVCC propagation	61
25	YAML Catalog Loader	62
25.1	Precedence	62
25.2	Validation	62
25.3	Diagnostics	62

VI	Transport Layer	63
26	The Canonical API Façade	64
26.1	State	64
26.2	Operations module	64
26.3	Session handle	64
27	DTOs	65
27.1	Module layout	65
27.2	Common patterns	65
27.3	Shape invariants	65
28	Transport Adapters	66
28.1	Tauri	66
28.2	HTTP	66
28.3	MCP	67
28.4	EventEmitter	67
29	Parity Harness	68
29.1	Phase A — DTO shape	68
29.2	Phase B — Scenario	68
29.3	Deferred Phase C	69
30	Session Persistence	70
30.1	On-disk layout	70
30.2	SessionFile shape	70
30.3	Load progress	70
30.4	v2 migration (deferred)	70
A	Complexity Summary	72
B	Catalog Reference	73
C	Benchmarks	74
C.1	Running	74
C.2	Current baseline (illustrative)	74
C.3	Three-way SIMD comparison (planned)	74
D	Source Index	75
D.1	graph_hunter_core/src/	75
D.2	graph_hunter_api/src/	76
D.3	app/src-tauri/	76
D.4	graph-hunter-mcp/	77
D.5	libgraphmatch/	77

Preface

This document describes the architecture and algorithms of GraphHunter, a temporal-graph threat-hunting engine. The style is modelled on *Introduction to Algorithms* (Cormen, Leiserson, Rivest, Stein — “CLRS”): each algorithm is presented as formal pseudocode with line numbers, followed by a derivation of its time and space complexity and, where the invariants aren’t obvious, a correctness argument.

The reference is internal: its purpose is to let a future maintainer (or the author, six months from now) understand *why* the engine is shaped the way it is without reading every source file. Every chapter ends with an *In the code* block giving `file:line` pointers so readers can drop into the Rust implementation on any point of interest.

Parts I–III are fleshed out in this revision. Parts IV–VI (ingest pipeline, extension seams, transport layer) ship as stubs with structures and file references but without full pseudocode; the intent is to expand them chapter-by-chapter in follow-up revisions.

Part I

Preliminaries

Chapter 1

Introduction

1.1 Problem domain

GraphHunter is an interactive threat-hunting tool built around a single idea: most attacker behaviour is naturally a path through a directed temporal graph, so let the analyst *ask for paths* rather than write per-table SQL. A hypothesis is expressed in a small arrow-chain DSL — for example,

```
User -[Auth \{status="Failure"\}] -> IP HAVING count(distinct IP) >= 3 WITHIN 24h
```

and the engine returns the paths in an ingested graph that satisfy it, ranked by composite anomaly score. The graph itself is built from heterogeneous log sources (Microsoft Sentinel, Defender XDR, Fortinet, Sysmon, IIS, generic CSV/JSON). Typical workloads range from a few thousand edges (one incident ticket) to hundreds of millions (multi-week EVTX dump from a medium enterprise).

1.2 System overview

The codebase is a Rust monorepo with five top-level components, layered as in Figure 1.1.

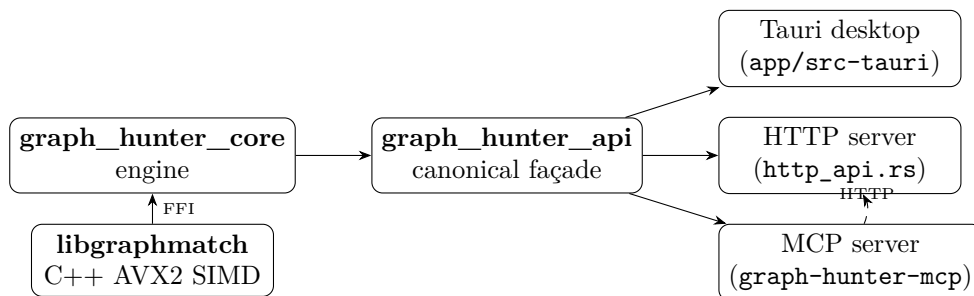


Figure 1.1: Crate graph. The engine (**graph_hunter_core**) is consumed by a canonical API façade (**graph_hunter_api**); three transport layers delegate to that façade. The MCP server reaches the same engine by calling the HTTP server over the loopback interface rather than linking to the façade directly.

The engine is a pure Rust library: no framework dependencies, no global state beyond a handful of `OnceLock`-cached catalogs. The façade crate (**graph_hunter_api**) wraps the engine with session management, DTOs, and an `EventEmitter` trait, so the three transport layers can share a single implementation without importing the low-level types themselves. The C++ SIMD backend (**libgraphmatch**) is linked via FFI when the `simd` feature is enabled; a pure-Rust `std::arch` alternative (**simd_rust**) is shipping as its replacement (Chapter 11).

1.3 What this reference covers

The document walks the engine bottom-up:

- **Part II** — the data structures the engine stores in memory: compact relations, the bidirectional string interner, the in-memory graph, the spillable edge store, the external metadata store, and the hypothesis AST the DSL parser produces.
- **Part III** — the core algorithms: DSL parsing, temporal pattern matching (DFS with pruning, smart anomaly-bounded variant), SIMD acceleration, anomaly scoring, centrality analytics (Brandes betweenness, temporal PageRank), path deduplication & pagination, and hunt diff.
- **Part IV** — the streaming ingest pipeline: `LogSource` trait, parsers, the bounded-channel writer, MVCC edge-level stamping, and overflow persistence.
- **Part V** — the extension seams introduced by the P2 refactor: `GraphRead/GraphWrite`, `ScoreComponent/Scorer`, `StringInternerBackend`, `IngestAdapter`, and the YAML catalog loader.
- **Part VI** — the transport layer: the canonical API, DTOs, `EventEmitter`, Tauri/HTTP/MCP adapters, the parity harness, and session persistence.

Four appendices close the document: a one-page complexity summary (App. A), the ATT&CK catalog reference (App. B), a benchmark table (App. C), and a source index mapping every algorithm to its `file:line` (App. D).

In the code.

Crate roots are `graph\_hunter\_core/Cargo.toml`, `graph\_hunter\_api/Cargo.toml`, `app/src-tauri/Cargo.toml`, `graph-hunter-mcp/package.json`, and `libgraphmatch/CMakeLists.txt`.

Chapter 2

Notation and Preliminaries

This chapter fixes the notation used throughout the document. The conventions are CLRS-compatible where possible.

2.1 Graphs

Definition 2.1 (Temporal directed multigraph). A *temporal directed multigraph* is a tuple $G = (V, E, \tau, \ell_V, \ell_E)$ where V is a finite set of vertices, $E \subseteq V \times V \times \mathbb{N}$ is a multiset of directed edges (the third component is a unique edge identifier, so multiple edges between the same pair are distinct elements of E), $\tau : E \rightarrow \mathbb{Z}$ assigns each edge an event-time timestamp, $\ell_V : V \rightarrow \mathcal{T}_V$ labels vertices with an entity type, and $\ell_E : E \rightarrow \mathcal{T}_E$ labels edges with a relation type.

We will write $|V| = n$ and $|E| = m$. The out-neighbourhood of a vertex $v \in V$ at edge time $[t_0, t_1]$ is

$$N_{[t_0, t_1]}^+(v) = \{u \in V \mid \exists e = (v, u, k) \in E : \tau(e) \in [t_0, t_1]\}.$$

The in-neighbourhood $N_{[t_0, t_1]}^-(v)$ is defined symmetrically. When the time window is the full domain we drop the subscript and just write $N^+(v)$, $N^-(v)$.

Definition 2.2 (MVCC extension). The *MVCC extension* of G adds a second timestamp $\iota : E \rightarrow \mathbb{Z}$ recording when each edge was inserted into the graph. The engine stores ι as a compact delta against a fixed epoch; see Chapter 3. A snapshot query at instant s considers only the subgraph

$$G|_s = (V, \{e \in E \mid \iota(e) \leq s\}, \tau, \ell_V, \ell_E),$$

i.e. “what the graph knew at time s .”

2.2 Paths and hypotheses

Definition 2.3 (Temporal path). A *temporal path* of length ℓ in G is a sequence

$$\pi = (v_0, e_1, v_1, e_2, \dots, e_\ell, v_\ell),$$

with $v_i \in V$, $e_i = (v_{i-1}, v_i, k_i) \in E$, and $\tau(e_1) \leq \tau(e_2) \leq \dots \leq \tau(e_\ell)$ (temporal monotonicity). We will frequently abbreviate a path as its vertex list $(v_0, v_1, \dots, v_\ell)$ when the intermediate edges are clear from context.

Definition 2.4 (k -simple path). A path is *k -simple* iff no vertex appears more than k times in it. k -simplicity relaxes the classical requirement of no-repeats ($k = 1$) to allow cycles that a real attacker exhibits — for instance, a C2 callback loop where the same process re-enters the path. k is set per hypothesis; the default is 1.

Definition 2.5 (Hypothesis). A *hypothesis* H is an arrow chain

$$H = (\sigma_0, \rho_1, P_1^{\text{edge}}, P_1^{\text{dst}}, \sigma_1, \rho_2, \dots, \rho_\ell, P_\ell^{\text{edge}}, P_\ell^{\text{dst}}, \sigma_\ell)$$

where $\sigma_i \in \mathcal{T}_V \cup \{*\}$ is the required entity type of the i -th vertex (or $*$ for “any”), $\rho_i \in \mathcal{T}_E \cup \{*\}$ is the required relation type of the i -th edge, and $P_i^{\text{edge}}, P_i^{\text{dst}}$ are optional predicates on edge metadata and destination-vertex metadata respectively. The chain also carries aggregation constraints (HAVING, WITHIN) processed post-match. A path $\pi = (v_0, \dots, v_\ell)$ *satisfies* H iff $\ell_V(v_i) \in \sigma_i$, $\ell_E(e_i) \in \rho_i$, and every predicate holds for the respective edge or destination.

2.3 Asymptotic notation

Standard CLRS conventions apply. $O(f(n))$ is upper bound, $\Theta(f(n))$ tight bound, $\Omega(f(n))$ lower bound. “Amortized $O(f)$ ” means the per-operation cost averaged over a worst-case sequence is $O(f)$, even if individual operations are more expensive.

2.4 Memory model

Rust’s ownership model makes most invariants locally checkable. Where the engine relies on interior mutability, we call it out explicitly — `Arc<RwLock<_>` for shared session state, `RwLock<SpillableEdgeStore>` for the in-memory edge buffer that may be flushed to disk during ingest, `OnceLock` for the process-wide catalog snapshot. The engine never shares mutable graphs across threads without a lock; the DFS matcher parallelizes by giving each rayon worker an immutable reference and a thread-local scratch stack (Chapter 10).

2.5 Source-pointer convention

Every chapter ends with an *In the code* block listing `file:line` pointers into the Rust implementation. The paths are relative to the repository root, e.g. `graph\_hunter\_core/src/graph.rs:461`. Line numbers drift across revisions, so prefer grepping the function name if the pointer is stale.

Part II

Core Data Structures

Chapter 3

The CompactRelation Encoding

The single densest data structure in the engine is the in-memory edge representation. A graph of 200 M edges at ~200 bytes each (the full `Relation` struct with metadata, strings, et cetera) would be 40 GB — so the engine stores every edge twice: once as a rich `Relation` used only while the edge is being added, and then as a 32-byte `CompactRelation` that lives in the adjacency list for the remainder of the session.

3.1 Layout

`CompactRelation` is `\#[repr(C)]`, fields ordered largest-alignment first to avoid any interior padding. Figure 3.1 shows the byte layout.

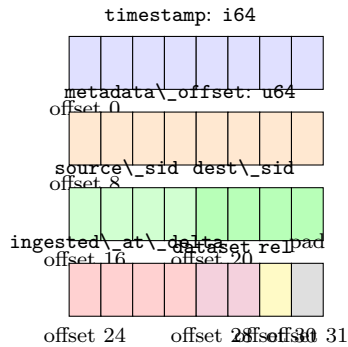


Figure 3.1: Memory layout of `CompactRelation` (32 bytes). The u32 `ingested_at_delta` (P3) reclaims 4 of the 5 bytes of trailing padding in the pre-P3 layout.

Semantically:

- `timestamp`: event time of the edge, epoch seconds.
- `metadata_offset`: offset into the external `MetadataStore` (Chapter 7); zero means “no metadata.”
- `source_sid`, `dest_sid`: 32-bit handles into the `StringIntern` (Chapter 4).
- `ingested_at_delta`: seconds since the MVCC epoch base (see below); zero on legacy edges.
- `dataset_tag`: 16-bit handle into the per-graph `Arc<str>` dataset identifier table; zero means “no dataset.”
- `rel_type_tag`: 8-bit enum tag for the relation type.

The `rel\_type\_tag` is deliberately `u8` so the matcher can do branchless SIMD comparisons (Chapter 11).

3.2 MVCC encoding

Invariant (MVCC size). `size\_of::<CompactRelation>() == 32`. This is asserted in `tests/relation\_schema.rs`. Violating it costs $\sim 20\%$ of the in-memory graph footprint, so the assertion is load-bearing.

The MVCC delta is measured against `INGEST\_EPOCH\_BASE = 1 704 067 200` seconds (2024-01-01 UTC). A `u32` covers $\approx 2^{32}$ seconds, or about 136 years; the engine saturates at `u32::MAX` on ingest beyond ~ 2160 . Legacy edges loaded from a pre-P3 session file resolve to the epoch base itself, which is a defined if imprecise moment.

Algorithm 3.1 shows the encoder. Decoding is the trivial `INGEST\_EPOCH\_BASE + delta as i64`.

Algorithm 3.1: ENCODE-INGESTED-AT

Input: absolute epoch-second timestamp t
Output: `u32` delta to store in `CompactRelation`

```

1 if  $t \leq \text{INGEST\_EPOCH\_BASE}$  then
2   | return 0;
3 end
4  $\delta \leftarrow t - \text{INGEST\_EPOCH\_BASE}$ ;
5 if  $\delta > \text{u32::MAX}$  then
6   | return u32::MAX ;                                ▷ saturate
7 end
8 return  $\delta$  as u32;
```

Complexity.

Constant time and space. Storage per edge is 32 bytes, giving $\Theta(32m)$ for the edge list of a graph with m edges.

3.3 Why not `\#[repr(C, packed)]`?

A packed layout would shave another 5 bytes by eliminating the trailing alignment pad, reaching 27 bytes per edge ($\sim 16\%$ additional saving). The engine does not use it because:

1. Every load of a packed field requires a byte-wise copy to aligned storage to satisfy the reference-alignment invariant in Rust. The DFS matcher reads `source\_sid` and `dest\_sid` millions of times per hunt; the copy overhead would dominate any memory saving.
2. The 32-byte alignment matches the AVX2 load unit, so SIMD scans of the edge list stay cache-line-friendly.

The trailing pad therefore stays as future real estate for further MVCC extensions (e.g., confidence score, source-tag bit flags).

In the code.

`graph\_hunter\_core/src/relation.rs:83` — struct definition.
`graph\_hunter\_core/src/relation.rs:151` — `encode\_ingested\_at` helper.
`graph\_hunter\_core/tests/mvcc\_ingested\_at.rs` — MVCC round-trip tests.
`graph\_hunter\_core/src/relation.rs` module tests — 32-byte size invariant.

Chapter 4

The String Interner

Entity identifiers in ingested logs are long and massively repetitive — a single `SecurityEvent` table in Sentinel will mention the same host name hundreds of thousands of times. Storing those strings as owned `String` inside `CompactRelation` would balloon the edge list. The engine side-steps this with a bidirectional interner that maps each unique string to a 32-bit handle `StrId`.

4.1 Structure

```
pub struct StrId(u32);

pub struct StringInterner {
    map:      HashMap<Arc<str>, StrId>,    // reverse lookup
    strings: Vec<Arc<str>>,                // forward lookup by index
}
```

Both containers hold `Arc<str>` pointing at the same heap-allocated slice, so each intern costs exactly one allocation (the `Arc::from(&str)` inside the `intern` path) plus two refcount bumps. The alternative — `String` in the map, plain `String` or a second copy in the vector — would double the allocator pressure.

Invariant (Intern bijection). For every index $i \in [0, \text{strings.len}())$ we have `map[strings[i]] == StrId(i as u32)`. The two containers are populated atomically inside `intern`; a panic during insertion cannot leave them out of sync because a half-inserted entry is not visible to subsequent lookups (the `map.insert` is the last action on the happy path).

4.2 Algorithms

Complexity.

INTERN is $O(1)$ amortized; the `HashMap` uses ahash which hits very few collisions on real entity IDs, and the `Vec::push` amortises. The heap allocation occurs on the miss path only. RESOLVE is $\Theta(1)$: a bounds check and a pointer deref. Space consumption is

$$\Theta\left(\sum_{s \in \mathcal{U}} |s| + \Theta(|\mathcal{U}|)\right)$$

where $\mathcal{U}$ is the set of unique strings — the first term is the heap content, the second is the $O(1)$ -bytes-per-entry overhead of the two containers.

Algorithm 4.1: INTERN

Input: interner I , string s
Output: StrId assigned to s

```

1 if  $s \in I.map$  then
2   | return  $I.map[s]$ ;
3 end
4  $id \leftarrow I.strings.len()$  as u32;
5  $arc \leftarrow Arc::from(s)$ ;                                ▷ 1 heap alloc
6  $I.strings.push(arc.clone())$ ;                                ▷ refcount++
7  $I.map.insert(arc, StrId(id))$ ;                                ▷ refcount++
8 return StrId(id);

```

Algorithm 4.2: RESOLVE

Input: interner I , StrId x
Output: $\&str$ for x , or $\perp$ if out-of-range

```

1 if  $x.0 \geq I.strings.len()$  then
2   | return  $\perp$ ;
3 end
4 return  $\&*I.strings[x.0]$ ;

```

4.3 Metrics

The engine exposes interner memory usage via `StringInterner::metrics()`:

```

pub struct InternerMetrics {
    pub unique_strings: usize,
    pub approx_bytes:    usize,    // sum of Arc<str>.len()
}

```

These are the trigger metrics for the deferred *generational* or *partitioned* back-ends (Chapter 23): when `unique\_strings` crosses $\sim 10^7$, or when a long-running streaming session’s `approx\_bytes` grows monotonically past a few hundred MB, the operator knows to switch.

4.4 The trait seam

The P2-F refactor introduced `StringInternerBackend` as the swap-in point:

```

pub trait StringInternerBackend {
    fn intern(&mut self, s: &str) -> StrId;
    fn resolve(&self, id: StrId) -> &str;
    fn try_resolve(&self, id: StrId) -> Option<&str>;
    fn get(&self, s: &str) -> Option<StrId>;
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool;
    fn reserve(&mut self, additional: usize);
    fn metrics(&self) -> InternerMetrics;
}

```

`StringInterner` is the only impl today. When the engine acquires evidence that the global-interner assumption breaks (multi-tenant process, 24×7 streaming past 10^7 unique strings), a `GenerationalInterner` or `PartitionedInterner` drops behind the same trait; the 121+ existing call sites keep compiling because they hold a concrete `StringInterner` and the inherent `metrics()` method delegates to the trait body.

In the code.

graph_hunter_core/src/interner.rs:1 — struct + trait + metrics.
graph_hunter_core/src/interner.rs:115 — unit tests pinning the bijection.

Chapter 5

The In-Memory Graph

The central struct is `InMemoryGraph` (aliased as `GraphHunter` for backward compatibility). It is the sole producer of `CompactRelation`, the sole owner of the `StringInterner` and `MetadataStore`, and the sole target of the DFS matcher.

5.1 Fields

```
pub struct InMemoryGraph {
    // Forward storage.
    pub(crate) entities:      HashMap<StrId, Entity>,
    pub(crate) edge_store:    SpillableEdgeStore,
    pub(crate) interner:      StringInterner,
    pub(crate) meta_store:    MetadataStore,

    // Indexes -- denormalised for query speed.
    pub(crate) reverse_adj:   HashMap<StrId, Vec<StrId>>,
    pub(crate) type_index:    HashMap<EntityType, HashSet<StrId>>,
    pub(crate) entity_type_tags: Vec<u8>, // dense by StrId

    // Dataset interner (Arc<str>, 16-bit tag).
    pub(crate) dataset_tags:  Vec<Arc<str>>,
    pub(crate) dataset_map:   HashMap<Arc<str>, u16>,
}
```

The redundancy between `entities[...].entity_type` and `entity\_type\_tags[...]` is deliberate: the former is the canonical source but costs a `HashMap` lookup; the latter is an SoA array indexed by `StrId.0 as usize` — a constant-time byte read that the matcher uses inside its inner loop to reject edges whose destination type doesn't match the hypothesis step.

Invariant (Index coherence). For every `StrId x` produced by `interner.intern(\_)`: `entities.contains\_key(x)` iff the string has been added as an entity, and in that case `type\_index[et].contains(\&x)` where `et = entities[\&x].entity\_type`, and `entity\_type\_tags[x.0 as usize]` equals that entity type's numeric tag.

5.2 Insertion algorithms

Complexity.

ADD-ENTITY: $O(1)$ amortized in the number of existing entities; the resize of `entity\_type\_tags` is amortized by the vector doubling strategy. ADD-RELATION: $O(1)$ amortized, dominated by `append` on the spillable store (Chapter 6) and the interner lookup. Space per inserted edge is 32 bytes plus any metadata bytes appended to `MetadataStore`.

Algorithm 5.1: ADD-ENTITY

Input: graph G , entity $e = (\text{id}, \text{type})$

```

1  $x \leftarrow G.\text{interner}.\text{intern}(e.\text{id});$ 
2 if  $x \in G.\text{entities}$  then
3   | return ; ▷ idempotent on dup
4 end
5  $G.\text{entities}.\text{insert}(x, e);$ 
6  $G.\text{type\_index}[\text{e.type}].\text{insert}(x);$ 
7  $G.\text{entity\_type\_tags}.\text{resize}(\max(\text{len}, x.0+1), \_);$ 
8  $G.\text{entity\_type\_tags}[x.0] \$\text{gets} \$ \text{tag}(\text{e.type});$ 
9  $G.\text{reverse\_adj}.\text{entry}(x).\text{or\_default}();$ 
10 return;
```

Algorithm 5.2: ADD-RELATION

Input: graph G , relation $r = (\text{src}, \text{dst}, \text{rel}, \tau, M)$

```

1  $s \leftarrow G.\text{interner}.\text{get}(r.\text{src});$  fail if  $\text{None}$ ;
2  $d \leftarrow G.\text{interner}.\text{get}(r.\text{dst});$  fail if  $\text{None}$ ;
3  $o \leftarrow G.\text{meta\_store}.\text{append}(M);$ 
4  $t \leftarrow \text{ENCODE-INGESTED-AT}(\text{NOW-SECONDS}());$ 
5  $c \leftarrow \text{CompactRelation}(s, d, \text{rel}, \tau, o, \text{dsTag}, t);$ 
6  $G.\text{edge\_store}.\text{append}(s, c);$ 
7  $G.\text{reverse\_adj}[d].\text{push}(s);$ 
8 return;
```

5.3 Finalization

The adjacency lists inside `edge_store` are not sorted by timestamp on insertion — sorting on every `ADD-RELATION` would pay $O(\log d)$ for every insert and we bulk-insert from streaming sources. Instead, after a batch, the caller invokes `sort_edges_by_timestamp`, which is the external trigger for `SpillableEdgeStore::finalize`:

Algorithm 5.3: SORT-EDGES-BY-TIMESTAMP

Input: graph G

```

1  $G.\text{edge\_store}.\text{finalize}();$  ▷ k-way merge, §6
2 for  $s \in G.\text{entities}.\text{keys}$  do
3   |  $\text{sort } G.\text{edge\_store}.\text{get\_edges}(s) \text{ in place by } (\text{timestamp}, \text{dest\_sid});$ 
3   |   ▷  $O(d(s) \log d(s))$ 
4 end
```

Complexity.

$O(m \log \bar{d})$ where $\bar{d}$ is the average out-degree, since each adjacency list of length $d(v)$ is sorted independently. If the store was in spill mode, `finalize` first does a k -way merge at $O(m \log k)$ cost (Chapter 6). Space grows only by the mergefile size during this step.

In the code.

```

graph\_hunter\_core/src/graph.rs:67 — struct fields.
graph\_hunter\_core/src/graph.rs:211 — ADD-ENTITY.
graph\_hunter\_core/src/graph.rs:227 — ADD-RELATION.
graph\_hunter\_core/src/graph.rs:437 — SORT-EDGES-BY-TIMESTAMP.
```

Chapter 6

The Spillable Edge Store

Ingesting a 28 GB EVTX dump on a laptop with 16 GB of RAM requires spilling in-memory structures to disk before the process is OOM- killed. `SpillableEdgeStore` implements that for the edge list: edges accumulate in an in-memory buffer until a watermark is crossed; past the watermark, the buffer is sorted and written to a sorted “run” file on disk, and a fresh buffer is started. At query time the store presents a single logical adjacency list by merging all runs on finalize.

6.1 Modes

Definition 6.1 (Store mode). The store is either *in-memory* (no runs on disk) or *spilled* (one or more runs on disk). A third “finalized” state is reached after `finalize` has merged all runs into a single mmap-backed file; once finalized, the store is read-only.

Fields:

```
struct SpillableEdgeStore {
    buffer:          Vec<CompactRelation>,
    buffer_index:    HashMap<StrId, Vec<u32>>, // positions in buffer
    runs:            Vec<SpillRunHandle>,      // on-disk sorted runs
    merged_mmap:     Option<MmapFile>,         // finalized state
    finalized:        bool,
    spill_limit:      usize,                   // edges before flush
}
```

`buffer\_index[s]` carries the *positions* (u32 indexes) of source `s`’s edges inside the current buffer, not the edges themselves. This is what makes `get\_edges(s)` cheap in in-memory mode: the caller gets a slice sliced by those positions, without a HashMap-of-Vecs copy.

6.2 Append

Algorithm 6.1: SPILL-APPEND

Input: store S , source s , compact c

```
1 if  $S.buffer.len() \geq S.spill\_limit$  then
2   | FLUSH-BUFFER-TO-RUN( $S$ );                                ▷ see below
3 end
4  $p \leftarrow S.buffer.len()$ ;
5  $S.buffer.push(c)$ ;
6  $S.buffer\_index[s].push(p)$ ;
```

Algorithm 6.2: FLUSH-BUFFER-TO-RUN

Input: store S

- 1 sort S .buffer lexicographically by (source_sid, timestamp, dest_sid);
- 2 write the sorted buffer to a temp file f as raw bytes (headered with count and version);
- 3 S .runs.push(handle(f));
- 4 clear S .buffer and S .buffer_index;

Complexity.

SPILL-APPEND is $O(1)$ amortized. Every `spill_limit` appends triggers a FLUSH of cost $O(L \log L)$ where $L = \text{spill_limit}$, distributed across the ingest stream. The total spill-mode cost to ingest N edges is $O(N \log L + N/L \cdot L) = O(N \log L)$ — logarithmic in the flush size, linear in N .

6.3 Finalization via k -way merge

After all appends are done, `finalize` produces a single on-disk sorted stream. If no flushes occurred the path is trivial: sort the in-memory buffer, done. Otherwise the engine opens cursors on every run plus a cursor over the sorted in-memory buffer and merges them with a min-heap:

Algorithm 6.3: K-WAY-MERGE

Input: cursors $[c_0, \dots, c_{k-1}]$, output file f

- 1 Build a binary min-heap H over the cursors, keyed by their head element (source_sid, timestamp, dest_sid);
- 2 **while** $H \neq \emptyset$ **do**
- 3 $c \leftarrow H.\text{pop_min}()$;
- 4 write c 's head element to f ;
- 5 $c.\text{advance}()$;
- 6 **if** c is non-empty **then**
- 7 $H.\text{push}(c)$
- 8 **end**
- 9 **end**

Complexity.

Each of the N elements is popped and pushed from the heap exactly once, paying $O(\log k)$ per operation, for a total of $O(N \log k)$ time. The heap holds at most k cursors at any moment, so RAM usage is $O(k)$ regardless of N — this is the property that lets the engine process a 28 GB stream in 16 GB of RAM. The merged output is memory-mapped into the process for subsequent reads, so `get_edges(s)` remains $O(d(s))$.

Theorem 6.2 (Spill memory bound). *At any point during ingestion the memory footprint of `SpillableEdgeStore` is at most `spill_limit · size_of::CompactRelation` plus a few KB of bookkeeping.*

Proof. SPILL-APPEND flushes the buffer whenever its length reaches `spill_limit`; between flushes the buffer holds at most `spill_limit` elements, each 32 B, plus the index `HashMap<StrId, Vec<u32>>` which totals $O(L)$ entries. The on-disk runs do not occupy RAM until merged. The merge itself only holds k heap cursors, bounded by the number of flushes; each cursor pre-buffers a small IO block (a few KB). □ □

6.4 Reads

Algorithm 6.4: GET-EDGES

Input: store S , source s
Output: slice of `CompactRelation` for s , empty if unknown

```

1 if  $S.finalized$  and  $S.merged\_mmap$  is Some then
2   | locate  $s$ 's start offset via the on-disk index ;           ▷ bsearch  $O(\log n)$ 
3   | return slice of length count from the mmap ;
4 end
5 if  $\neg S.finalized$  then
6   | return positions from S.buffer\_index[$s$] materialized from S.buffer ;
   |   ▷  $O(d(s))$ 
7 end

```

Complexity.

$O(\log n + d(s))$ in the finalized case, $O(d(s))$ in the non-finalized case (plus any work to copy positions into a slice).

In the code.

`graph\_hunter\_core/src/spill.rs` — struct + all algorithms.
`graph\_hunter\_core/src/spill.rs:327` — `finalize` entry point.
`graph\_hunter\_core/src/spill.rs:524` — `get\_edges`.
`graph\_hunter\_core/benches/dedup\_throughput.rs` — spill ingest benchmark.

Chapter 7

The Metadata Store

Per-edge metadata (arbitrary `HashMap<String, String>` key-value pairs populated by parsers) is the most variable-size field associated with a relation. Inlining it into `CompactRelation` would force a variable-length record and destroy the 32-byte invariant (Chapter 3). The engine instead stores metadata externally in `MetadataStore` and keeps only a 64-bit offset in `CompactRelation`.

7.1 Structure

`MetadataStore` is an append-only byte buffer with a simple length-prefixed encoding:

```
pub struct MetadataStore {
    buf: Vec<u8>,    // append-only
}
// layout of one record at offset o:
//   u32 num_entries
//   repeated num_entries times:
//     u16 key_len, key bytes, u16 value_len, value bytes
```

The first valid record offset is 1, so `metadata\_offset = 0` in `CompactRelation` is unambiguously “no metadata.” On `append`, the store returns the offset at which it wrote the new record and that offset lives in `CompactRelation` for the remainder of the session.

7.2 Operations

Algorithm 7.1: METADATA-APPEND

Input: store M , map $K : \{(k, v)\}$
Output: offset $o \geq 1$, or 0 if K is empty

- 1 **if** K is empty **then**
- 2 **return** 0;
- 3 **end**
- 4 $o \leftarrow M.\text{buf}.\text{len}() + 1$;
- 5 write `u32(len($K$))` then each (key, value) pair with `u16` length prefixes into $M.\text{buf}$;
- 6 **return** o ;

Complexity.

METADATA-APPEND: $O(|K| + \sum |k| + \sum |v|)$. METADATA-READ: same. Space per edge is zero if the parser produced no metadata, otherwise the byte length of the encoded record. A typical Sysmon edge produces ~ 60 bytes of metadata, compared with the 32 bytes of `CompactRelation`

Algorithm 7.2: METADATA-READ

Input: store M , offset o
Output: `HashMap<String, String>`, or empty if $o = 0$

```

1 if  $o = 0$  then
2   | return empty map;
3 end
4 position cursor at  $o - 1$  in  $M.\text{buf}$ ;
5 read u32  $\text{num}$ ;
6 for  $i \leftarrow 0$  to  $\text{num} - 1$  do
7   | read u16  $k\_len$ , next  $k\_len$  bytes as UTF-8  $\rightarrow k$ ;
8   | read u16  $v\_len$ , next  $v\_len$  bytes as UTF-8  $\rightarrow v$ ;
9   | insert  $(k, v)$ ;
10 end
11 return map;
```

— which is why the external store matters: the edges stay packed in the hot matcher path and metadata is paid for only when an analyst asks.

7.3 Immutability

Invariant (Metadata append-only). Once a record has been written to `MetadataStore.buf`, no operation truncates or rewrites the preceding bytes. Offsets are therefore stable for the lifetime of the graph. In particular, the matcher can capture an offset into a `CompactRelation` at ingest time and read the metadata through it at hunt time without worrying about concurrent writer mutation.

When a session is persisted the entire `MetadataStore.buf` is serialised alongside the entity/relation lists; on load the store is reconstituted verbatim, preserving every offset.

In the code.

`graph\_hunter\_core/src/metadata\_store.rs` — struct and append/read routines.
`graph\_hunter\_core/src/graph.rs:246` — where `ADD-RELATION` calls `meta\_store.append`.

Chapter 8

The Hypothesis AST

Every hunt is driven by a parsed `Hypothesis` value. The AST is deliberately small — a flat vector of steps with per-step predicate lists, plus optional aggregation and time-window clauses attached to the whole chain.

8.1 Types

```
pub struct Hypothesis {
    pub name: String,
    pub steps: Vec<HypothesisStep>,
    pub k_simplicity: usize, // default 1
    pub aggregations: Vec<AggClause>, // HAVING clauses
    pub window_seconds: Option<u64>, // WITHIN clause
}

pub struct HypothesisStep {
    pub origin_type: EntityType,
    pub relation_type: RelationType,
    pub dest_type: EntityType,
    pub edge_predicates: Vec<Predicate>, // on edge metadata
    pub dest_predicates: Vec<Predicate>, // on destination-vertex
    pub origin_binding: Option<String>, // '(User u)' for ref in HAVING
    pub dest_binding: Option<String>,
}

pub enum Predicate {
    Eq { key: String, value: String },
    Substr { key: String, pattern: String },
    Regex { key: String, pattern: String },
    In { key: String, values: Vec<String> },
}

pub struct AggClause {
    pub op: AggOp, // Count, CountDistinct, Min, Max, Sum, Avg
    pub column: AggColumn, // bound name or step index
    pub cmp: AggComparison, // >=, <=, >, <, ==
    pub value: f64,
}
```

8.2 Invariants

Invariant (Step count ≥ 1). A hypothesis must have at least one step; the DSL parser rejects the empty input with `ApiError::InvalidInput`. This keeps the matcher from having to treat the zero-edge case as a special branch.

Invariant (k -simplicity positive). $k_{\text{simplicity}} \geq 1$. Zero would admit only the empty path and violates the sense of “max visits per vertex.”

Invariant (Binding uniqueness). If `origin\_binding` or `dest\_binding` is `Some(b)` on any step, b does not repeat on any other binding position inside the hypothesis. The parser enforces this at parse time so the matcher can look bindings up by name during aggregation without disambiguation.

8.3 Static compatibility check

Before running a hunt, the engine can cross-reference the hypothesis against the graph’s relation schema to see if any step is structurally impossible on the loaded dataset — e.g., a `User-[Auth]->Host` step against a purely cloud-identity graph where no `Host` entities exist. This is `CHECK-CATALOG-COMPATIBILITY` (Chapter 25), which returns a `CatalogEntryWithStatus` flagging the missing entity or relation types per step.

Algorithm 8.1: CHECK-STEP-COMPATIBILITY

Input: schema Σ (multiset of (source, rel, target) triples), step s
Output: bool compatible, set of missing axes

```

1 for each  $(src, r, dst) \in \Sigma$  do
2   if  $(src = s.origin\_type \vee s.origin\_type = *)$ 
      $\wedge (r = s.relation\_type \vee s.relation\_type = *) \wedge (dst = s.dest\_type \vee s.dest\_type = *)$ 
     then
3     return (true,  $\emptyset$ );
4   end
5 end
6 missing  $\leftarrow$  set of axes absent from any triple in  $\Sigma$  ;
7 return (false, missing);
```

Complexity.

$O(|\Sigma|)$ per step, $O(|\Sigma| \cdot \ell)$ per hypothesis, where ℓ is the step count. $|\Sigma|$ is bounded by $|\mathcal{T}_V|^2 \cdot |\mathcal{T}_E|$ in practice (there are only a few dozen entity/relation types), so the check is effectively constant.

In the code.

`graph\_hunter\_core/src/hypothesis.rs` — `Hypothesis, HypothesisStep, Predicate`.
`graph\_hunter\_core/src/catalog/mod.rs:105` — `check\_catalog\_compatibility`.

Part III

Core Algorithms

Chapter 9

DSL Parsing

The DSL is a small arrow-chain grammar designed to be read aloud (“User authenticates to a Host and executes a Process”) and to fit on one line of a UI input. The parser is a hand-written single-pass recursive-descent walker — no grammar file, no parser-generator dependency.

9.1 Grammar

```
hypothesis      := step ( step )* modifier*
step             := entity_type arrow
arrow           := "[" rel_type predicates? "]"->" entity_type binding?
                  predicates? dest_predicates?
entity_type      := ident ( "(" binding ")" )? | "*"
rel_type        := ident | "*"
predicates       := "{" predicate ( "," predicate )* "}"
predicate       := key ("=" | "~" | " in ") (literal | list)
modifier        := having | within | k_clause
having          := "HAVING" agg_op "(" agg_arg ")" cmp literal
within          := "WITHIN" int time_unit
k_clause        := "{k=" int "}"
```

Every token type is recognised by a character-class probe, so the parser performs exactly one pass over the input string. There is no backtracking; the grammar is LL(1) after lexer-level lookahead for predicates.

9.2 Algorithm

Complexity.

$O(|\text{src}|)$ time and space. Every token is visited exactly once; the `HashMap`-based predicate lists are $O(1)$ to populate. Space is linear in the parsed chain length plus the predicate literals.

Theorem 9.1 (Parser determinism). *PARSE-DSL is deterministic: for any fixed input string, it produces the same *Hypothesis* or the same *DslError* on every invocation.*

Proof. No hash-map iteration order, no random values, no calls to the system clock anywhere in the parser. All data structures built during parsing (the final `edge\_predicates` and `dest\_predicates` on each step) preserve insertion order. □ □

In the code.

`graph\_hunter\_core/src/dsl.rs:47` — `parse\_dsl` entry point.
`graph\_hunter\_core/src/dsl.rs` — full grammar implementation (private helpers).

Algorithm 9.1: PARSE-DSL

Input: string `src`, optional hypothesis name `nm`
Output: Hypothesis or DslError

```

1 toks  $\leftarrow$  TOKENIZE(src) ;  $\triangleright O(|\text{src}|)$ 
2 steps  $\leftarrow \emptyset$ ;
3  $\sigma_0 \leftarrow$  PARSE-ENTITY-TYPE(toks);
4 while toks has remaining “ $\rightarrow$ ” do
5    $\rho, P^{\text{edge}} \leftarrow$  PARSE-EDGE-SPEC(toks);
6    $\sigma_i \leftarrow$  PARSE-ENTITY-TYPE(toks);
7    $P^{\text{dst}} \leftarrow$  PARSE-DEST-PREDICATES(toks);
8   push HypothesisStep( $\sigma_{i-1}, \rho, \sigma_i, P^{\text{edge}}, P^{\text{dst}}$ ) onto steps;
9 end
10  $k \leftarrow$  parse optional {k=N}, default 1;
11  $A \leftarrow$  PARSE-HAVING(toks);
12  $w \leftarrow$  PARSE-WITHIN(toks);
13 return Hypothesis(nm, steps,  $k$ ,  $A$ ,  $w$ );
```

Chapter 10

Temporal Pattern Matching

The matcher is the engine’s hot path. Every hunt the analyst runs, every `run\_hunt` call from Claude, every CLI invocation ends up here. The implementation ships three coordinated variants:

- `SEARCH-TEMPORAL-PATTERN` — the entry point; dispatches work to rayon workers and collects results.
- `DFS-MATCH-ITERATIVE` — the baseline iterative DFS executed per starting vertex.
- `DFS-MATCH-SMART` — an anomaly-bounded variant that maintains a top- K min-heap and prunes branches whose running anomaly estimate makes them inadmissible.

A fourth path, `SEARCH-TEMPORAL-PATTERN-SIMD`, delegates to `libgraphmatch` or the pure-Rust `simd\_rust` module (Chapter 11) and is gated on the `simd` feature plus a runtime CPU probe.

10.1 Shared state

```
// Scratch allocated once per worker and reused across starts.
struct DfsFrame {
    sid: StrId,
    step_idx: usize,
    edge_hi: usize, // upper bound of outgoing edges
    edge_cursor: usize, // next edge index to try
}

struct SmartDfsFrame {
    sid: StrId,
    step_idx: usize,
    edge_hi: usize,
    edge_cursor: usize,
    path_anomaly_sum: f64, // sum of node_anomaly_estimate along path
}
```

Stack frames are plain structs stored in a `Vec<Frame>` that is reused across starting vertices. This is the single biggest allocation savings in the matcher — the old recursive implementation allocated per-frame on the system stack and could not trivially restart.

Invariant (k -simplicity). The running `visit\_count: HashMap<StrId, usize>` maintained inside every DFS walk satisfies `visit\_count[v] <= k` for every v on the current path, where k is `hypothesis.k\_simplicity`. Violating this would admit paths that revisit a vertex more than k times, contradicting the hypothesis’ semantic. The algorithm enforces the invariant at push-time: if incrementing `visit\_count[dest]` would exceed k , the edge is skipped.

10.2 Entry point: Search-Temporal-Pattern

The outer algorithm partitions work by starting vertex. For each entity whose type matches H 's first step, a rayon-managed worker thread runs DFS-MATCH-ITERATIVE on that starting vertex with its own scratch stack and result buffer. When every worker has finished, results are merged, scored, deduplicated and paginated (Chapter 14).

Algorithm 10.1: SEARCH-TEMPORAL-PATTERN

Input: graph G , hypothesis H , optional window (t_0, t_1) , optional cap max
Output: vector of path vectors

```

1 starts  $\leftarrow$  G.type\_index[H.steps[0].origin\_type];
2 if starts =  $\emptyset$  then
3   | return  $\emptyset$ ;
4 end
5 cap  $\leftarrow$  AtomicUsize(0);
6 results  $\leftarrow$  parallel-map over starts: foreach  $s \in$  starts Input: parallel
7 do
8   | local  $\leftarrow$  DFS-MATCH-ITERATIVE( $G, H, s, t_0, t_1, \text{cap}, \text{max}$ ) ;
9   | return local;
10 end
11 return concat of results;
```

The AtomicUsize cap is read by every worker on every candidate push so that once the aggregate result count reaches max, remaining workers stop as early as possible. This is a classical early-exit pattern: the count may briefly exceed max because workers commit then check, but the overshoot is bounded by the number of rayon workers (typically 8–16).

10.3 The iterative DFS

PUSH-EDGES-FOR-STEP is a one-liner that binary-searches `edge\_store[v]` for the first edge with `timestamp` $\geq t_0$ and sets `edge\_hi` to the first edge with `timestamp` $> t_1$. The sort by timestamp performed at finalize (Chapter 5) is what makes this binary search correct.

Theorem 10.1 (Matcher correctness). *Upon return, DFS-MATCH-ITERATIVE produces exactly the set of temporal paths of length ℓ rooted at s_0 that satisfy the hypothesis H within $[t_0, t_1]$ and respect k -simplicity.*

Proof sketch. By induction on the hypothesis depth. The base case ($\ell = 1$) is the first push into `stack`; the check is step-by-step equivalent to the hypothesis constraints on the first edge. The inductive step observes that each admitted edge extends the path by exactly one vertex, updates `visit` consistently, and that the cycle check rejects exactly those extensions that would exceed k -simplicity. The stack unwinds restoring `visit` and the path, so every branch is explored independently; no path is ever omitted because of missed backtracking. $\square$ $\square$

Complexity.

The worst case is a graph where every edge matches every step. Then from a start s_0 the DFS visits up to $\bar{b}^\ell$ paths (where $\bar{b}$ is the average out-degree of qualifying edges per step and ℓ is the step count), giving $O(\bar{b}^\ell \cdot \ell)$ time per start. Across all n_s candidate starts, the total is

$$T_{\text{match}}(G, H) = O(n_s \cdot \bar{b}^\ell \cdot \ell).$$

Space per worker is $O(\ell)$ for the stack and path plus $O(\ell \cdot k)$ for the `visit` map (each entry is one vertex, and the path holds at most ℓ distinct vertices each visited at most k times).

Algorithm 10.2: DFS-MATCH-ITERATIVE

Input: graph G , hypothesis H , start s_0 , window (t_0, t_1) , shared cap counter cap , $\text{cap}_{\max}$

Output: list of matching paths rooted at s_0

```

1 results  $\leftarrow \emptyset$ ; stack  $\leftarrow \emptyset$ ; path  $\leftarrow [s_0]$ ; visit  $\leftarrow \{s_0 : 1\}$ ;
2 PUSH-EDGES-FOR-STEP(stack,  $G$ ,  $s_0$ , step 0,  $t_0, t_1$ );
3 while stack  $\neq \emptyset$  do
4    $f \leftarrow$  top of stack ;
5   if  $f.\text{edge\_cursor} \geq f.\text{edge\_hi}$  then
6     | pop  $f$ ; pop last vertex from path; decrement visit[.]; continue;
7   end
8    $c \leftarrow G.\text{edge\_store}[f.\text{sid}][f.\text{edge\_cursor}]$ ;
9    $f.\text{edge\_cursor} \leftarrow f.\text{edge\_cursor} + 1$ ;
10  // filter: relation type
11  if  $c.\text{rel\_type\_tag} \neq \text{step}[f.\text{step\_idx}].\text{rel}$  then
12    | continue;
13  end
14  // filter: destination entity type via SoA tag
15   $u \leftarrow c.\text{dest\_sid}$ ;
16  if  $G.\text{entity\_type\_tags}[u.0] \neq \text{step}[f.\text{step\_idx}].\text{dst}$  then
17    | continue;
18  end
19  // filter: time window
20  if  $c.\text{timestamp} \notin [t_0, t_1]$  then
21    | continue;
22  end
23  // filter: edge predicates (on metadata)
24  if  $\text{EVALUATE-PREDICATES}(\text{step}.\text{edge\_predicates}, c, G) = \text{false}$  then
25    | continue;
26  end
27  // cycle prevention via  $k$ -simplicity
28  if  $\text{visit}[u] + 1 > H.k$  then
29    | continue;
30  end
31  visit[ $u$ ]  $\leftarrow$  visit[ $u$ ] + 1; append  $u$  to path;
32  if  $f.\text{step\_idx} + 1 = \text{len}(H.\text{steps})$  then
33    | record path in results;
34    | increment cap atomically; if  $\text{cap} \geq \text{cap}_{\max}$  then
35      | break loop;
36    | end
37    | visit[ $u$ ]  $\leftarrow$  visit[ $u$ ] - 1; pop last vertex; continue;
38  end
39  push new frame:  $(u, f.\text{step\_idx} + 1, \dots)$  onto stack;
40  PUSH-EDGES-FOR-STEP(stack,  $G$ ,  $u$ ,  $f.\text{step\_idx} + 1$ ,  $t_0, t_1$ );
41 end
42 return results;

```

With W workers total matcher space is $O(W \cdot \ell \cdot k)$. In practice $\bar{b}$ is tiny because the type-tag fast-path eliminates most outgoing edges on the first byte comparison; empirical results from `benches/hunt\_latency.rs` are in App. C.

10.4 The smart variant

DFS-MATCH-SMART is used when the caller requests the top- K most anomalous paths instead of “all paths.” It differs from the iterative variant only in the candidate ordering and the result accumulator:

- **Accumulator:** a min-heap of size at most K ordered by composite anomaly score. When full, new paths are accepted iff they score higher than the heap’s minimum.
- **Running bound:** each frame tracks `path\_anomaly\_sum`, the sum of `node\_anomaly\_estimate` (Chapter 12) along the path so far. Extending the path cannot decrease this sum; if the sum already falls below the heap’s minimum, the branch is pruned.

Algorithm 10.3: MERGE-INTO-TOPK

Input: local heap L , global heap G , capacity K

```

1 foreach  $p \in L$  do
2   if  $|G| < K$  then
3      $G.\text{push}(p)$ 
4   end
5   else if  $\text{score}(p) > \min(G)$  then
6      $G.\text{pop\_min}(); G.\text{push}(p);$ 
7   end
8 end
```

Complexity.

$O((|L| + |G|) \log K)$. When the matcher joins rayon workers, each local heap of size $\leq K$ merges into a single global heap at total cost $O(WK \log K)$, negligible compared to the DFS work.

10.5 Parallelism

The matcher is embarrassingly parallel in the starting vertex. Because each worker’s DFS reads the graph exclusively (all writers are blocked on an `RwLock::write` at the session level), there is no inter-worker synchronisation beyond the shared `cap` counter and the final reduction into a single result vector.

Theorem 10.2 (Parallel speedup). *Given W rayon workers and n_s starting vertices with approximately equal DFS work per start, the matcher achieves a speedup factor of $\min(W, n_s)$ up to the constant cost of the final reduction.*

The proof is Brent’s theorem applied to the trivial partition: each start’s DFS has no inter-start dependencies, and the final reduction is $O(\text{total results})$ which is typically orders of magnitude smaller than the DFS itself.

In the code.

`graph\_hunter\_core/src/graph.rs:461` — `search\_temporal\_pattern`.
`graph\_hunter\_core/src/graph.rs:1121` — `dfs\_match\_iterative`.

graph_hunter_core/src/graph.rs:782 — search_temporal_pattern_smart.
graph_hunter_core/src/graph.rs:932 — dfs_match_smart.
graph_hunter_core/benches/hunt_latency.rs — performance benchmarks.

Chapter 11

SIMD Acceleration

The DFS matcher’s inner work is dominated by edge filtering: checking `rel\_type\_tag`, `entity\_type\_tags`, timestamp inclusion, and predicate evaluation for millions of candidate edges per hunt. Three of those four checks are byte-level comparisons that vectorise cleanly. GraphHunter has two SIMD paths:

1. **libgraphmatch** — a C++ library linked via FFI when the `simd` feature is compiled and the host CPU reports AVX2 support. Peak throughput around 3× the Rust scalar matcher.
2. **Pure-Rust `simd\_rust` (P2-D)** — a `std::arch::x86\_64` implementation that uses AVX2 intrinsics without leaving the Rust toolchain. Currently implements one primitive, `INTERSECT-SORTED-U32`, with more on the way.

11.1 libgraphmatch FFI bridge

```
// graph_hunter_core/src/simd_matcher.rs (abridged)
unsafe extern "C" {
    fn gm_graph_new() -> *mut c_void;
    fn gm_graph_add_node(g: *mut c_void, id: u32, et: u8);
    fn gm_graph_add_edge(g: *mut c_void, src: u32, dst: u32, rt: u8, ts: i64);
    fn gm_graph_finalize(g: *mut c_void);
    fn gm_matcher_run(m: *mut c_void, max_results: u32, min_score: f64) -> *mut c_void;
    fn gm_results_free(r: *mut c_void);
    fn gm_graph_free(g: *mut c_void);
}
```

The Rust side wraps the `*mut c\_void` handles in `GmGraph` (RAII, drop calls `gm\_graph\_free`) and caches the resulting graph alongside a `HashMap<String, u32>` that maps string IDs to the C++ node indexes (see `CachedSimdGraph` in `simd\_matcher.rs`). The cache survives across hunts on the same session; it’s invalidated whenever the graph is mutated.

`SEARCH-TEMPORAL-PATTERN-SIMD` is therefore a short function:

The fallback to the Rust smart matcher is unavoidable because `libgraphmatch` does not evaluate metadata predicates (`\{key="val"\}`) — those require `MetadataStore` access which the C++ side cannot do without shipping the raw bytes across FFI.

11.2 Pure-Rust AVX2 intersection

The P2-D refactor began replacing `libgraphmatch` one primitive at a time. The first one shipped is `simd\_rust::intersect\_sorted\_u32\_count`: given two ascending-sorted deduplicated

Algorithm 11.1: SEARCH-TEMPORAL-PATTERN-SIMD

Input: graph G , hypothesis H , cap max

```

1 if runtime AVX2 not detected or  $H$  has predicates then
2   | return SEARCH-TEMPORAL-PATTERN-SMART( $G, H, \dots$ ) ;           ▷ fallback
3 end
4  $C \leftarrow G.\text{cached\_simd\_graph}()$  ;                           ▷ build once, reuse
5  $r \leftarrow gm\_matcher\_run(\$C, \text{max}, 0.0)$ ;
6 return translate\_node\_ids( $\$r, \$C.\text{text \{string\_map\}}$ );

```

`&[u32]` slices, return the count of common elements. This is the primitive that `libgraphmatch` uses for neighbor intersection during SIMD pattern expansion, and it's the cheapest primitive to prove safe in isolation.

The algorithm is the classical “broadcast-and-compare” schema used throughout SIMD intersection literature. Its correctness follows from the invariant that j is monotonically increased only after the entire $L[j..j + 8]$ window has been ruled out (every element strictly less than the current s); because both S and L are sorted, no later s can have a match at any index $< j$.

Theorem 11.1 (Intersect complexity). *On inputs of sizes $|A|$ and $|B|$ with $|S| = \min(|A|, |B|)$, $|L| = \max(|A|, |B|)$:*

- *Time: $O(|S| \cdot (|L|/8 + 1))$ in the worst case. The $/8$ factor is the AVX2 lane width. In practice the short-side iteration amortises closer to $O(|S| + |L|/8)$ because j slides forward monotonically.*
- *Space: $O(1)$ auxiliary (the register state).*

Theorem 11.2 (Runtime safety). *INTERSECT-SORTED-U32-COUNT does not invoke undefined behaviour even when compiled without AVX2, because the unsafe intrinsic function is only called through `is\_x86\_feature\_detected!("avx2")` at runtime. The compile-time `#[target\_feature(enable = "avx2")]` gate only enables the intrinsic body inside a function the scalar path does not call.*

Proof sketch. The public wrapper is safe. It checks `is\_x86\_feature\_detected!("avx2")` first; only on a hit does it invoke the unsafe function body. When AVX2 is not reported, the scalar merge is called and no AVX2 instructions are executed. The `target\_feature` gate means the unsafe function body itself is compiled with AVX2 codegen regardless, but that's fine because it's only ever called when the CPU supports it. □ □

11.3 Roadmap

Remaining primitives to port from `libgraphmatch` to `simd\_rust`:

- `count\_edges\_by\_rel\_type`: scan a `&[CompactRelation]` for entries matching a given `rel\_type\_tag`. Trivial (`_mm256_cmpeq_epi8` on the packed tag array).
- `timestamp\_range\_filter`: mark edges whose `timestamp` falls in $[t_0, t_1]$. Requires 64-bit comparisons — one extra instruction per lane.
- `intersect\_sorted\_u32\_out`: the output-materialising variant of the current count-only primitive.

Once all three land, SEARCH-TEMPORAL-PATTERN-SIMD's `libgraphmatch` path can be replaced with a pure-Rust dispatch, retiring the C++ dependency.

Algorithm 11.2: INTERSECT-SORTED-U32-COUNT

Input: sorted slices A, B of `u32`, both ascending and deduplicated
Output: count of common elements

```

1 if  $|A| < 32$  or  $|B| < 32$  then
2   | return SCALAR-MERGE( $A, B$ );
3 end
4 if AVX2 not detected at runtime then
5   | return SCALAR-MERGE( $A, B$ );
6 end
   // short side is  $S$ , long side is  $L$ 
7 if  $|A| \leq |B|$  then  $(S, L) \leftarrow (A, B)$ ;
8 else  $(S, L) \leftarrow (B, A)$ ;
9  $j \leftarrow 0$ ;  $\text{count} \leftarrow 0$ ;
10 foreach  $s \in S$  do
11   | while  $j + 8 \leq |L|$  do
12     |  $\text{chunk} \leftarrow$  AVX2 load  $L[j..j + 8]$ ;
13     |  $\text{needle} \leftarrow$  AVX2 broadcast  $s$  to 8 lanes;
14     |  $\text{cmp} \leftarrow \_mm256\_cmpeq\_epi32(\text{chunk}, \text{needle})$ ;
15     |  $\text{mask} \leftarrow \_mm256\_movemask\_epi8(\text{cmp})$ ;
16     | if  $\text{mask} \neq 0$  then
17       |  $\text{count} \leftarrow \text{count} + 1$ ; break inner loop;
18     | end
   // slide  $j$  if every lane in chunk was less than  $s$ 
19     | if  $L[j + 7] < s$  then
20       |  $j \leftarrow j + 8$ ; continue;
21     | end
22     | break inner loop;
23   | end
24   | SCAN-TAIL( $L, j, s, \text{count}$ ) ;           ▷ scalar finish for trailing  $< 8$  elements
25 end
26 return  $\text{count}$ ;

```

In the code.

`graph\_hunter\_core/src/simd\_matcher.rs` — FFI bridge.
`graph\_hunter\_core/src/simd\_rust.rs` — INTERSECT-SORTED-U32-COUNT.
`libgraphmatch/src/simd\_intersect.cpp` — C++ reference implementation.
`graph\_hunter\_core/src/graph.rs:689` — SEARCH-TEMPORAL-PATTERN-SIMD.

Chapter 12

Anomaly Scoring

Path ranking is the second half of what the engine does: after the matcher returns all paths satisfying a hypothesis, the `AnomalyScorer` assigns each path a composite score in $[0, 1]$ and optional per-component breakdown so the analyst can triage. The scorer is a weighted average of five dimensions (entity rarity, edge rarity, neighbourhood concentration, temporal novelty, and GNN threat), each computed as an average over the vertices of the path.

12.1 State and finalisation

```
pub struct AnomalyScorer {
    entity_freq:      HashMap<String, u64>,
    pair_freq:        HashMap<(String, String), u64>,
    first_seen:       HashMap<String, i64>,
    nc_cache:         HashMap<String, f64>,
    gnn_cache:        HashMap<String, f64>,
    log_max_freq:     f64,
    log_max_pair_freq: f64,
    tau_min:          i64,
    tau_max:          i64,
    weights:          ScoringWeights,
    finalized:        bool,
}
```

The scorer is populated incrementally during ingest (`observe\_entity`, `observe\_edge`); those are $O(1)$ amortised and do not compute anything derived. After ingest, `finalize` converts the raw counts into the normalisation factors the per-path scoring needs.

Algorithm 12.1: ANOMALY-FINALIZE

Input: scorer S , graph G

- 1 $S.\text{log_max_freq} \leftarrow \ln(\max \text{entity_freq.values})$;
- 2 $S.\text{log_max_pair_freq} \leftarrow \ln(\max \text{pair_freq.values})$;
- 3 $(\tau_{\min}, \tau_{\max}) \leftarrow (\min, \max)$ of the non-zero values of `first\_seen.values`;
- 4 $S.\text{nc_cache} \leftarrow \emptyset$;
- 5 **foreach** $v \in G.\text{entities}$ **do**
- 6 $S.\text{nc_cache}[v] \leftarrow \text{COMPUTE-NC}(v, G)$;
- 7 **end**
- 8 $S.\text{finalized} \leftarrow \text{true}$;

Complexity.

$O(V + E + \sum_v d(v)) = O(V + E)$ for the counters. The Shannon-entropy cache costs an

additional $O(\sum_v d(v)) = O(E)$. Space is $O(V)$ for the caches plus whatever the raw counters already consumed.

12.2 Neighbourhood concentration

Algorithm 12.2: COMPUTE-NC

Input: vertex v , graph G
Output: neighbourhood concentration $\text{NC}(v) \in [0, 1]$

```

1 counts  $\leftarrow \emptyset$ ; total  $\leftarrow 0$ ;
2 foreach outgoing edge  $(v, u) \in G$  do
3   | counts[ $\ell_V(u)$ ] += 1; total += 1;
4 end
5 foreach incoming edge  $(w, v) \in G$  do
6   | counts[ $\ell_V(w)$ ] += 1; total += 1;
7 end
8  $K \leftarrow |\text{counts}|$ ;
9 if  $K \leq 1$  then
10  | return 1.0;
11 end
12  $H \leftarrow 0$ ;
13 foreach  $c \in \text{counts.values}$  do
14  |  $p \leftarrow c/\text{total}$ ;
15  |  $H \leftarrow H - p \log_2 p$ ;
16 end
17  $H_{\max} \leftarrow \log_2 K$ ;  $\text{NC} \leftarrow 1 - H/H_{\max}$ ;
18 return clamp( $\text{NC}$ , 0, 1);

```

NC is 1 when the neighbourhood is maximally homogeneous (a single entity-type family) and 0 when maximally diverse; the clamp guards floating-point drift near the extremes.

Complexity.

$O(d(v))$ time, $O(|\mathcal{T}_V|)$ space (neighbour type counts only; small constant).

12.3 Scoring a path

Three of the four non-GNN components are simple averages over the path’s vertices; the edge-rarity component averages over the path’s edges.

Each component is individually clamped to $[0, 1]$ so that a stale cache entry (extremely rare, but possible during a re-finalize race) cannot make a component negative. The weight renormalisation means the caller can set weights in any absolute scale.

Complexity.

$O(\ell)$ time, $O(1)$ auxiliary space. The expensive state was built by ANOMALY-FINALIZE; per-path scoring is cheap.

12.4 The trait seam (P2-A)

The refactored scoring module (`graph\_hunter\_core/src/scoring/`) makes each component a separate `impl ScoreComponent`:

Algorithm 12.3: SCORE-PATH

Input: path $\pi = (v_0, \dots, v_\ell)$, scorer S
Output: composite score c , breakdown b

- 1 **if** π empty or $\neg S.finalized$ **then**
- 2 | **return** $(0, 0)$;
- 3 **end**
- // Component 1: entity rarity
- 4 $ER \leftarrow \frac{1}{\ell+1} \sum_v \left(1 - \frac{\ln(\text{entity_freq}[v])}{S.\text{log_max_freq}}\right)$;
- // Component 2: edge rarity
- 5 $\text{EdgeR} \leftarrow \frac{1}{\ell} \sum_{i=1}^{\ell} \left(1 - \frac{\ln(\text{pair_freq}[v_{i-1}, v_i])}{S.\text{log_max_pair_freq}}\right)$;
- // Component 3: NC average
- 6 $NC \leftarrow \frac{1}{\ell+1} \sum_v S.\text{nc_cache}[v]$;
- // Component 4: temporal novelty
- 7 $TN \leftarrow \frac{1}{\ell+1} \sum_v \frac{\text{first_seen}[v] - \tau_{\min}}{\tau_{\max} - \tau_{\min}}$;
- // Component 5: GNN threat
- 8 $GT \leftarrow \frac{1}{\ell+1} \sum_v S.\text{gnn_cache}.get(v).\text{unwrap_or}(0)$;
- // composite via renormalised weights
- 9 $W \leftarrow \sum \text{weights}; c \leftarrow \frac{1}{W} \sum_i w_i \cdot \text{component}_i$;
- 10 **return** $(\text{clamp}(c, 0, 1), \text{breakdown})$;

```
pub trait ScoreComponent: Send + Sync {
    fn name(&self) -> &'static str;
    fn score_path(&self, ctx: &PathContext<'_>) -> f64;
    fn explain(&self, value: f64, ctx: &PathContext<'_>) -> Option<String>
        {
            None
        }
}
```

CompositeScorer holds a `Vec<Box<dyn ScoreComponent>>` plus matching weights and applies SCORE-PATH-style aggregation. Adding a sixth component (Bayesian, learned model, etc.) is a new struct + one `vec!{} push`.

Theorem 12.1 (v1/v2 equivalence). *For every weight vector w and every path π , `AnomalyScorer::score_path` and `AnomalyScorer::score_path_v2`(w, π) agree to within $8 \cdot \epsilon_{f64}$.*

This is verified by `tests/scoring_snapshot.rs`. The equivalence is the precondition for eventually retiring the hardcoded v1 path.

12.5 Node-level fast estimate

During smart-DFS pruning (Chapter 10), we need an $O(1)$ estimate of a single vertex’s anomaly so the running `path_anomaly_sum` can reject branches. That’s `node_anomaly_estimate`:

```
pub fn node_anomaly_estimate(&self, v: &str) -> f64 {
    if !self.finalized { return 0.5; }
    let er = /* Component 1 for a single vertex */;
    let nc = self.nc_cache.get(v).copied().unwrap_or(1.0);
    let tn = /* Component 4 for a single vertex */;
    let gt = self.gnn_cache.get(v).copied().unwrap_or(0.0);
    combine_with_weights(er, nc, tn, gt, &self.weights)
}
```

Edge rarity isn't part of the estimate because the DFS hasn't yet chosen the next edge when it queries the bound. The estimate is therefore a loose upper bound on the node's contribution to the composite; it's admissible (never over-prunes) provided the weights are non-negative, which the engine enforces at config load time.

In the code.

```
graph\_hunter\_core/src/anomaly.rs:148 — finalize.  
graph\_hunter\_core/src/anomaly.rs:191 — compute\_nc.  
graph\_hunter\_core/src/anomaly.rs:241 — score\_path (v1).  
graph\_hunter\_core/src/anomaly.rs:351 — node\_anomaly\_estimate.  
graph\_hunter\_core/src/scoring/ — trait-based v2.  
graph\_hunter\_core/tests/scoring\_snapshot.rs — v1/v2 equivalence pinning.
```

Chapter 13

Centrality and Analytics

Beyond hunt execution, the engine offers a handful of graph-wide analytics: betweenness centrality (for surfacing structurally important nodes), temporal PageRank (for identifying nodes whose influence peaked near a reference time), and a few auxiliary helpers such as `get\_relation\_schema`. This chapter covers the first two in depth — they are the non-trivial analytics and the ones most likely to surprise a reader with their cost.

13.1 Betweenness centrality (sampled Brandes)

Brandes’ 2001 algorithm computes exact betweenness on a directed graph in $O(VE)$ time, which is prohibitive for the multi-M-edge graphs the engine regularly ingests. GraphHunter uses a sampled variant: pick a uniformly random subset of S starting vertices, run single-source shortest-path BFS from each, and approximate betweenness as the average contribution across the sample.

Algorithm 13.1: COMPUTE-BETWEENNESS

Input: graph G , sample size S
Output: map $B : V \rightarrow \mathbb{R}_{\geq 0}$

```
1 if  $|V| \leq S$  then
2   | sources  $\leftarrow V$ 
3 end
4 else
5   | sources  $\leftarrow$  uniform sample of  $S$  vertices
6 end
7  $B \leftarrow \{v : 0 \ \forall v \in V\};$ 
8 foreach source  $s \in$  sources do
9   | BRANDES-SINGLE-SOURCE( $G, s, B$ );
10 end
11 scale every entry of  $B$  by  $|V|/|\text{sources}|$ ;
12 return  $B$ ;
```

Complexity.

$O(V + E)$ time and space per source (standard BFS + back-pass). Total time with S sources is $O(S(V + E))$; for the default $S = 500$ and a graph with $V = 10^5$ and $E = 10^6$, the cost is $\sim 5 \times 10^8$ operations — a few hundred ms on a modern laptop. Space is $O(V + E)$: the BFS state is reused across sources.

The expected approximation quality is bounded by Chernoff on a random sample: with $S = O(\log V / \epsilon^2)$ sources, per-vertex betweenness is estimated within multiplicative factor $1 \pm \epsilon$

Algorithm 13.2: BRANDES-SINGLE-SOURCE

Input: graph G , source s , accumulator B

// Phase 1: BFS forward

```

1 for every  $v$ :  $d[v] \leftarrow \infty$ ,  $\sigma[v] \leftarrow 0$ ,  $P[v] \leftarrow \emptyset$ ;
2  $d[s] \leftarrow 0$ ;  $\sigma[s] \leftarrow 1$ ; queue  $Q \leftarrow [s]$ ; stack  $St \leftarrow []$ ;
3 while  $Q$  nonempty do
4    $v \leftarrow Q.\text{pop\_front}$ ; push  $v$  onto  $St$ ;
5   foreach out-neighbour  $u$  of  $v$  (undirected view: outgoing  $\cup$  incoming) do
6     if  $d[u] = \infty$  then
7        $d[u] \leftarrow d[v] + 1$ ; push  $u$  onto  $Q$ 
8     end
9     if  $d[u] = d[v] + 1$  then
10       $\sigma[u] \leftarrow \sigma[u] + \sigma[v]$ ; append  $v$  to  $P[u]$ ;
11    end
12  end
13 end
  
```

// Phase 2: back-accumulation

```

14 for every  $v$ :  $\delta[v] \leftarrow 0$ ;
15 while  $St$  nonempty do
16    $w \leftarrow St.\text{pop}$ ;
17   foreach  $v \in P[w]$  do
18      $\delta[v] \leftarrow \delta[v] + \frac{\sigma[v]}{\sigma[w]}(1 + \delta[w])$ ;
19   end
20   if  $w \neq s$  then
21      $B[w] \leftarrow B[w] + \delta[w]$ ;
22   end
23 end
  
```

with high probability, assuming the shortest-path structure is not pathologically skewed. The engine does not attempt to certify this bound online; the sample size is a configurable knob.

13.2 Temporal PageRank

Temporal PageRank extends classical PageRank with an exponential decay on edge weights: an edge with timestamp $\tau(e)$ contributes to the rank computed at reference time τ_0 with weight $\exp(-\lambda(\tau_0 - \tau(e)))$ if $\tau(e) \leq \tau_0$, zero otherwise.

Algorithm 13.3: COMPUTE-TEMPORAL-PAGERANK

Input: graph G , decay λ , damping d , max iterations $I_{\max}$, tolerance ϵ , reference time τ_0
Output: map $r : V \rightarrow [0, 1]$ summing to 1

```

1  $n \leftarrow |V|$ ; initialise  $r[v] \leftarrow 1/n$  for all  $v \in V$ ;
2 compute edge weights  $w(e) \leftarrow \exp(-\lambda \max(0, \tau_0 - \tau(e)))$ ;
3 compute normalised outgoing weights  $w_{\text{out}}(v) \leftarrow \sum_{e:v \rightarrow \cdot} w(e)$ ;
4 for  $iter \leftarrow 1$  to  $I_{\max}$  do
5    $r' \leftarrow \{v : (1 - d)/n \ \forall v\}$ ;
6   foreach vertex  $v$  with  $w_{\text{out}}(v) > 0$  do
7     foreach outgoing edge  $e = (v \rightarrow u)$  do
8        $r'[u] \leftarrow r'[u] + d \cdot r[v] \cdot \frac{w(e)}{w_{\text{out}}(v)}$ ;
9     end
10  end
11  distribute dangling-node mass (vertices with  $w_{\text{out}} = 0$ ) uniformly over  $V$ ;
12  if  $\|r' - r\|_1 < \epsilon$  then
13    return  $r'$ ;
14  end
15   $r \leftarrow r'$ ;
16 end
17 return  $r$ ;
```

Complexity.

Each iteration is $O(V + E)$. Power iteration on a damped chain converges geometrically at rate d , so the iteration count is $O(\log(1/\epsilon)/\log(1/d))$. With $d = 0.85$ and $\epsilon = 10^{-6}$ the engine observes convergence in 10–20 iterations in practice, giving total time $O(20(V + E))$ — linear in graph size with a small constant.

13.3 Relation schema

`get\_relation\_schema` walks every edge and groups into unique triples $(\ell_V(\text{src}), \ell_E(e), \ell_V(\text{dst}))$, returning a vector sorted lexicographically. It is used by Chapter 25’s compatibility check.

Complexity.

$O(E + T \log T)$ where $T = |M|$ is the number of unique triples. T is tiny in practice (low hundreds) so the sort cost is nominal; the dominant cost is the linear edge walk.

In the code.

```

graph\_hunter\_core/src/analytics.rs:641 — compute\_betweenness.
graph\_hunter\_core/src/analytics.rs:746 — compute\_temporal\_pagerank.
graph\_hunter\_core/src/analytics.rs:1289 — get\_relation\_schema.
```

Algorithm 13.4: GET-RELATION-SCHEMA

Input: graph G

Output: sorted vector of (src_type, rel_type, dst_type, count)

```

1  $M \leftarrow$  empty HashMap;
2 foreach edge  $e = (u, v, k)$  in  $G$  do
3   |  $M[(\ell_V(u), \ell_E(e), \ell_V(v))]$   $+= 1$ ;
4 end
5 out  $\leftarrow M$ .into_iter as vector;
6 sort out lexicographically;
7 return out;
```

Chapter 14

Path Deduplication and Pagination

The matcher can return many paths for a lax hypothesis. Before an analyst sees them they're deduplicated (depending on the requested mode), scored, filtered, sorted, and paginated. `score\_and\_paginate` is the single function that does all of that; it lives in `graph\_hunter\_core/src/analytics.rs`.

14.1 Dedup modes

Definition 14.1 (Dedup modes). `DedupMode` is one of:

- **None** — every path from the matcher is kept.
- **ByPath** — paths with identical vertex sequences are merged into a single representative (the highest-scoring one).
- **ByEndpoints** — paths sharing the same (v_0, v_ℓ) pair are merged, regardless of the intermediate vertices.

ByPath is the default for hunts rendered in the UI: when the underlying graph has many parallel edges between the same pair of vertices, the matcher produces multiple identical-looking paths (one per edge), and the analyst only wants to see the topology once.

ByEndpoints is used during bulk analysis: “how many distinct start-end reachability pairs satisfy this hypothesis,” which is the right granularity for counting unique attack scenarios.

14.2 Algorithm

Complexity.

Let $P = |\Pi|$ (raw count) and $U = |\text{unique}|$ after dedup.

- Dedup in **ByPath/ByEndpoints** is $O(P \cdot \ell)$ (hashing the key of length ℓ); **None** is $O(P)$.
- Scoring is $O(U \cdot \ell)$ using `SCORE-PATH`.
- Sorting is $O(U \log U)$ comparisons.
- Slicing is $O(s)$.

Total: $O(P \cdot \ell + U \log U + s)$. Space is $O(U + s)$ above the input.

Algorithm 14.1: SCORE-AND-PAGINATE

Input: paths Π , page index p , page size s , optional min_score , mode M , scorer S
Output: page of paths + total count (pre-page)
 // Step 1: dedup according to mode

```

1 switch  $M$  do
2   case None do
3      $\text{unique} \leftarrow \Pi$ 
4   end
5   case ByPath do
6      $\text{key each } \pi \in \Pi \text{ by its vertex tuple; keep the highest-scoring } \pi \text{ per key}$ 
7   end
8   case ByEndpoints do
9      $\text{key each } \pi \in \Pi \text{ by } (v_0, v_\ell); \text{ keep the highest-scoring } \pi \text{ per key}$ 
10  end
11 end
    // Step 2: score
12 foreach  $\pi \in \text{unique}$  do
13    $(\text{composite}, \text{breakdown}) \leftarrow S.\text{score\_path}(\pi)$ 
14 end
    // Step 3: optional filter
15 if  $\text{min\_score}$  is Some then
16    $\text{drop } \pi \text{ with composite} < \text{min\_score}$ 
17 end
    // Step 4: sort DESC by (composite, max_node_score, total_score)
18  $\text{sort unique;}$ 
    // Step 5: slice
19  $\text{total} \leftarrow |\text{unique}|;$ 
20 return  $(\text{unique}[p \cdot s .. (p + 1) \cdot s], \text{total});$ 

```

14.3 Sort stability

The sort key is a tuple

$$(\text{composite}(\pi), \max_{v \in \pi} \text{composite}(\{v\}), \text{total_score}(\pi))$$

sorted in descending order. The three-way key matters because composite scores tie frequently on paths with short length — secondary and tertiary keys break ties deterministically. In particular the max-node score promotes paths that contain a single strongly anomalous vertex over paths that are uniformly mediocre.

14.4 Pagination contract

Invariant (Page stability). Two consecutive `get\_hunt\_page` calls with the same parameters return the same sorted order, provided the underlying graph has not been mutated. Path order is deterministic because every tie-breaker is a totally ordered quantity derived from the graph state.

The parity harness (Chapter 29) pins this invariant with a fixture that runs a hunt twice and asserts element-wise equality of the returned page.

In the code.

`graph\_hunter\_core/src/analytics.rs:1096` — `score\_and\_paginate\_paths`.
`graph\_hunter\_core/src/analytics.rs` — `DedupMode` variants and key derivation helpers.

Chapter 15

Hunt Diff

`diff\_hunts` answers the question “what changed between two points in time in the set of paths satisfying this hypothesis?” It is the foundation for the UI’s “what’s new since my last check” panel and for scheduled automation that flags freshly appeared attack chains.

15.1 Classical event-time variant

The baseline algorithm runs the hypothesis twice against the graph, bounding the time window at different upper-bound timestamps, and diffs the path sets.

Algorithm 15.1: DIFF-HUNTS

Input: graph G , hypothesis H , baseline timestamp t_{base} , current timestamp t_{cur} with $t_{\text{base}} < t_{\text{cur}}$
Output: (added: List[π], removed: List[π])

- 1 $\Pi_{\text{base}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN}(G, H, (0, t_{\text{base}}), \infty)$;
- 2 $\Pi_{\text{cur}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN}(G, H, (0, t_{\text{cur}}), \infty)$;
- 3 $U_{\text{base}} \leftarrow \text{SCORE-AND-PAGINATE}(\Pi_{\text{base}}, 0, \infty, \text{None}, \text{ByPath}).\text{unique}$;
- 4 $U_{\text{cur}} \leftarrow \text{same for } \Pi_{\text{cur}}$;
- 5 $K_{\text{base}} \leftarrow \text{set of path-keys in } U_{\text{base}}$;
- 6 $K_{\text{cur}} \leftarrow \text{same for } U_{\text{cur}}$;
- 7 $\text{added} \leftarrow U_{\text{cur}} \text{ filtered by key } \notin K_{\text{base}}$;
- 8 $\text{removed} \leftarrow U_{\text{base}} \text{ filtered by key } \notin K_{\text{cur}}$;
- 9 **return** (added, removed);

Complexity.

Two hunt executions plus a set-difference pass. If T_{match} is the matcher cost and P is the number of paths returned by either hunt, the total is $2T_{\text{match}} + O(P)$. Space is $O(P)$ above the matcher’s own scratch state.

15.2 Semantic gotcha: late-arriving data

The classical variant diffs by *event time*, not by *knowledge time*: an edge whose event timestamp is in the past but that was ingested after t_{base} will appear in both Π_{base} and Π_{cur} , because the upper bound on the time window doesn’t filter based on when the engine learned about the edge. This is the original motivation for the P3 MVCC work (Chapter 19): an analyst who closed a ticket at t_{base} may still want to know that a post-mortem ingest revealed new edges inside the window they already considered “done.”

15.3 The snapshot variant (deferred)

Once the P3 matcher extension (SEARCH-TEMPORAL-PATTERN-AT) lands, `diff\_by\_snapshot` becomes the richer alternative:

Algorithm 15.2: DIFF-BY-SNAPSHOT (deferred)

Input: graph G , hypothesis H , event window (t_0, t_1) , baseline snapshot ι_{base} , current snapshot ι_{cur}

Output: (added, removed) path lists

- 1 $\Pi_{\text{base}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN-AT}(G, H, (t_0, t_1), \iota_{\text{base}});$
 - 2 $\Pi_{\text{cur}} \leftarrow \text{SEARCH-TEMPORAL-PATTERN-AT}(G, H, (t_0, t_1), \iota_{\text{cur}});$
 - 3 ...set-diff as in DIFF-HUNTS ...
-

Key distinction: the event window (t_0, t_1) is fixed, and it's the knowledge snapshot ι that varies. The engine answers “what new paths in this event window have I learned about between ι_{base} and ι_{cur} ?” — the proper audit semantic for late-arriving enrichment, VirusTotal detonations that return after the event was ingested, threat-intel overlays that landed later, et cetera.

Complexity.

Same as the classical variant: $2T_{\text{match}} + O(P)$. The *value* delta is semantic, not performance.

15.4 Input validation

Invariant (Window ordering). `diff\_hunts` rejects $t_{\text{base}} > t_{\text{cur}}$ with `ApiError::InvalidInput`. A reversed window would produce an (added, removed) pair whose semantics are the user's responsibility to interpret; the engine prefers to fail loudly.

This invariant is exercised by the parity harness (`diff\_hunts\_rejects\_inverted\_window` in `graph\_hunter\_api/tests/parity/scenarios.rs`).

In the code.

`graph\_hunter\_core/src/analytics.rs:1342` — `diff\_hunts` (classical).
`graph\_hunter\_core/src/relation.rs:83` — the `ingested\_at\_delta` field that `diff\_by\_snapshot` will eventually consult.

Part IV

Ingestion Pipeline

Chapter 16

The LogSource Trait

Note. This chapter is a stub. The `LogSource` trait is the bottom layer of the streaming ingest pipeline: each implementation represents one external data producer (a Microsoft Sentinel workspace, a Defender XDR tenant, an Azure Data Lake blob source). A full walkthrough of the polling lifecycle — `poll\since` resumption semantics, cancellation, watermark monotonicity, backoff and quota interactions — is deferred to a follow-up revision.

16.1 Structure

```
#[async_trait]
pub trait LogSource: Send + Sync {
    fn source_id(&self) -> &str;
    fn kind(&self) -> SourceKind;
    fn available_tables(&self) -> &[&'static str];
    async fn check_auth(&self) -> Result<(), SourceError>;
    async fn query_scoped(
        &self,
        scope: QueryScope,
        cancel: CancellationToken,
    ) -> Result<ChunkStream, SourceError>;
    async fn poll_since(
        &self,
        table: &str,
        watermark: Watermark,
        cancel: CancellationToken,
    ) -> Result<PollStream, SourceError>;
}
```

A `RawChunk` is a `Bytes`-backed response fragment carrying `source\_id`, `table`, `fetch\_at`, and an optional `watermark\_candidate` the source suggests be persisted after successful parse.

16.2 Implementations

- `graph\_hunter\_core/src/sources/log\_analytics.rs` — Microsoft Sentinel / Azure Log Analytics.
- `graph\_hunter\_core/src/sources/defender\_xdr.rs` — Microsoft Defender XDR (Advanced Hunting).
- `graph\_hunter\_core/src/sources/data\_lake.rs` — Azure Data Lake blob enumeration.

- `graph\_hunter\_core/src/sources/aad\_training\_export.rs` — Azure AD training export pipeline.

In the code.

`graph\_hunter\_core/src/sources/mod.rs:275` — trait definition.
`graph\_hunter\_core/src/sources/token\_cache.rs` — Azure token caching.
`graph\_hunter\_core/src/sources/backoff.rs` — exponential backoff policy.

Chapter 17

Parsers

Note. Stub. The `LogParser` trait and its format-specific implementations (Sentinel JSON, Sysmon XML, Cognito, IIS W3C, FortiAnalyzer XML, generic CSV/JSON with field mappings) deserve a chapter each; this section lists the seam and the known implementations.

17.1 Trait

```
pub trait LogParser: Send + Sync {
    fn parse(&self, data: &str) -> Vec<ParsedTriple>;
    fn parse_rows(&self, rows: &[serde_json::Value]) -> Vec<ParsedTriple>
    {
        // Default: serialize rows back to a JSON array and delegate.
    }
}

pub type ParsedTriple = (Entity, Relation, Entity);
```

The `parse\_rows` override exists to short-circuit the “JSON $\rightarrow$ string $\rightarrow$ JSON” round-trip for parsers that already consume the `serde\_json::Value` shape (notably `SentinelJsonParser`).

17.2 Implementations

- `graph\_hunter\_core/src/sentinel.rs` — `SentinelJsonParser` (also overrides `parse\_rows`).
- `graph\_hunter\_core/src/sysmon.rs` — Sysmon XML event parsing.
- `graph\_hunter\_core/src/cognito.rs` — AWS Cognito audit parsing.
- `graph\_hunter\_core/src/iis\_w3c.rs` — IIS W3C log format.
- `graph\_hunter\_core/src/forti\_analyzer.rs` — FortiAnalyzer XML with per-handler registry.
- `graph\_hunter\_core/src/generic.rs` — generic CSV/JSON with a `FieldMapping` describing which column is which semantic role.
- `graph\_hunter\_core/src/csv\_parser.rs` — lightweight CSV helper used by `GenericParser`.

17.3 Parser dispatch

The engine does not do format auto-detection in the matcher path; instead, the ingest façade (`graph\_hunter\_api`) picks a parser per `RawChunk.encoding` (Json / Jsonl / Parquet) plus per-source registration. The preview family (`graph\_hunter\_core/src/preview.rs`, `field\_preview.rs`) lets the UI sniff a file and suggest a mapping before committing to ingest.

In the code.

`graph\_hunter\_core/src/parser.rs` — `LogParser` trait.

Chapter 18

Writer and Backpressure

Note. Stub. The writer task is the single point of mutation for the `InMemoryGraph` during streaming ingest. A full treatment (enrichment preemption, quota accounting, overflow to disk) is deferred.

18.1 Pipeline shape

`LogSource -> raw_channel -> ParserTask -> parsed_channel -> GraphWriter -> Graph`

Both channels are bounded `tokio::sync::mpsc` queues (default capacities 4 and 8 respectively, configurable per source). A full channel backpressures the upstream stage; a source whose poll cannot find room in `raw_channel` blocks on `send().await` until the writer has drained enough to make progress.

18.2 Modules

- `graph\_hunter\_core/src/ingest/batch.rs` — `ParsedBatch` unit of work (triples + source_id + dataset_id + priority + watermark).
- `graph\_hunter\_core/src/ingest/parser\_task.rs` — `ParserTask` consumes `raw_channel`, calls the format-specific `LogParser`, emits `ParsedBatch`.
- `graph\_hunter\_core/src/ingest/writer.rs` — `GraphWriter`: singleton task that drains `parsed_channel`, holds the graph's write lock for one batch at a time, and releases between batches so readers can make progress.
- `graph\_hunter\_core/src/ingest/source\_poller.rs` — `SourcePoller`: per-source timer loop driving `LogSource::poll\_since`.
- `graph\_hunter\_core/src/ingest/enrichment.rs` — on-demand enrichment batches, priority-preempted ahead of bulk ingest.
- `graph\_hunter\_core/src/ingest/channels.rs` — bounded channel constructors.
- `graph\_hunter\_core/src/ingest/overflow.rs` — serialises queued `ParsedBatches` to disk on cancellation; replays on restart (Chapter 20).

18.3 Lock contention

The writer holds `graph.write()` for the duration of a single `insert\_triples\_chunk` call and releases between batches. Readers (hunts, analytics queries, enrichment queries) acquire

`graph.read()` and coexist with each other. Typical batch sizes are 1–5 K triples, so a write hold lasts milliseconds — readers backing off once per batch is cheap.

Invariant (Writer uniqueness). Each session has at most one `GraphWriter` task active at any time. The `SessionStore` registers the writer’s `JoinHandle` when the task starts and refuses to start a second; this rules out writer-writer races entirely.

In the code.

`graph\_hunter\_core/src/ingest/writer.rs` — writer main loop.

Chapter 19

MVCC Edge-Level Stamping

Note. Stub. The P3 refactor introduced edge-level MVCC with zero memory overhead by reclaiming padding in `CompactRelation` (see Chapter 3 for the layout and Chapter 15 for the snapshot-aware diff semantics). This chapter will eventually describe the ingest pipeline’s stamping behaviour, the `search\_temporal\_pattern\_at` matcher extension, and the v1→v2 session-file / spill-file migration.

19.1 Current state

- `CompactRelation.ingested\_at\_delta`: u32 is stamped by `GraphHunter::add\_relation` to the current wall-clock second at insert time. Legacy edges (`delta = 0`) resolve to `INGEST\_EPOCH\_BASE`.
- `encode\_ingested\_at(ts)` is the ingest-path helper that clamps below the epoch base and saturates above `u32::MAX`.
- The 32-byte invariant on `CompactRelation` is CI-enforced (`relation::tests::compact\_relation\_stay`).

19.2 Follow-up work

- `SEARCH-TEMPORAL-PATTERN-AT( $G, H, (t_0, t_1), \iota$ )` — the matcher wrapper that filters edges by `ingested\_at() ≤  $\iota$` in addition to the event window. Expected 5–15 % cost bump on snapshot queries; transparent on current-view queries.
- `IngestAdapter` → writer propagation: forward `TripleBatch.fetched\_at` into `with\_ingested\_at` so streaming sources get their upstream timestamp rather than the local wall-clock. Deterministic for replay.
- `diff\_by\_snapshot` analytics function (Chapter 15).
- `SessionFile` `schema\_version` bump (v1→v2): backfill `ingested\_at` from `timestamp` for legacy edges, surface a UI banner “approximate snapshot”.
- Spill `SPILL\_VERSION` bump and CLI migrator.

In the code.

`graph\_hunter\_core/src/relation.rs:83` — `CompactRelation` layout with MVCC field.
`graph\_hunter\_core/src/graph.rs:227` — `add\_relation` stamping today.
`graph\_hunter\_core/tests/mvcc\_ingested\_at.rs` — end-to-end MVCC contract pinning.

Chapter 20

Overflow Persistence

Note. Stub. When the ingest pipeline is cancelled (user closes the session, process exit) or when the `parsed\_channel` fills faster than the writer can drain it, queued `ParsedBatches` are persisted to disk instead of being dropped. On the next session load the overflow is replayed in order before streaming resumes.

20.1 Invariants

- Every persisted batch retains its `source\_id`, `watermark`, and priority, so resumption advances the source's watermark exactly as a hot-path ingest would.
- Replay is idempotent: the graph is de-duplicating by $(source, dest, rel, timestamp)$ at `GraphWriter::insert\_triples\_chunk`, so the same batch applied twice has the same effect as once.

20.2 File format

Overflow batches are stored under the session directory as JSON files. The format is stable across restarts and deliberately human-readable: an operator can inspect queued work with `jq` without any tooling beyond a text editor.

In the code.

`graph\_hunter\_core/src/ingest/overflow.rs` — persistence and replay.

Part V

Extension Seams

Chapter 21

GraphRead / GraphWrite Traits

Note. Stub. The P2-B refactor extracted these traits so the engine’s consumers (anomaly scorer, GNN feature extractor, ingest writer) can depend on a semantic API rather than the concrete `GraphHunter` struct. The eventual goal is to swap in remote backends (Neo4j, a streaming MVCC store) without editing the scorer, matcher, or writer — provided the trait impls exist.

21.1 Traits

```
pub trait GraphRead {
    fn entity_count(&self) -> usize;
    fn relation_count(&self) -> usize;
    fn get_entity(&self, id: &str) -> Option<&Entity>;
    fn get_compact_relations(&self, src: &str)
        -> Cow<'_, [CompactRelation]>;
    fn get_reverse_source_ids(&self, id: &str) -> Vec<String>;
    fn entity_ids_for_type(&self, et: &EntityType) -> Option<Vec<String>>;
    fn entity_types_in_graph(&self) -> Vec<String>;
    fn materialize_relation(&self, c: &CompactRelation) -> Relation;
    fn supports_simd(&self) -> bool { false }
}

pub trait GraphWrite: GraphRead {
    fn add_entity(&mut self, e: Entity) -> Result<(), GraphError>;
    fn add_relation(&mut self, r: Relation) -> Result<(), GraphError>;
}
```

The `Cow<'_, [CompactRelation]>` return is the key to letting remote backends coexist: `InMemoryGraph` returns `Cow::Borrowed(&slice)` at zero cost; a remote impl returns `Cow::Owned(vec)` when it has to synthesise the list.

21.2 What’s on the trait vs off

On: semantic read and write, the operations a hypothetical remote backend would need to implement.

Off: the matcher itself (`search\_temporal\_pattern`), the SIMD dispatch, the analytics (`pagerank`, `betweenness`). These are engine-specific; a Neo4j backend would ship its own pattern engine rather than implement Rust’s DFS against a `\&dyn GraphRead`.

21.3 Dual-impl contract

`graph\_hunter\_core/tests/backend\_dual\_impl.rs` runs a generic `validate\_backend<B: GraphWrite>(b)` harness that exercises every trait method against `GraphHunter`. When a second impl lands, the same harness covers it for free.

In the code.

`graph\_hunter\_core/src/backend/mod.rs` — trait definitions.
`graph\_hunter\_core/src/backend/in\_memory.rs` — `impl GraphRead + GraphWrite` for `GraphHunter`.
`graph\_hunter\_core/tests/backend\_dual\_impl.rs` — dual-impl validation.

Chapter 22

Scorer and ScoreComponent Traits

Note. Stub. The P2-A refactor extracted each scoring dimension into a separate `impl` `ScoreComponent`. See Chapter 12 for the underlying math.

22.1 Traits

```
pub trait ScoreComponent: Send + Sync {
    fn name(&self) -> &'static str;
    fn score_path(&self, ctx: &PathContext<'_>) -> f64;
    fn explain(&self, value: f64, ctx: &PathContext<'_>) -> Option<String>
        {
            None
        }
}

pub struct ScoringCaches<'a> {
    pub entity_freq: &'a HashMap<String, u64>,
    pub pair_freq: &'a HashMap<(String, String), u64>,
    pub first_seen: &'a HashMap<String, i64>,
    pub nc_cache: &'a HashMap<String, f64>,
    pub gnn_cache: &'a HashMap<String, f64>,
    pub log_max_freq: f64,
    pub log_max_pair_freq: f64,
    pub tau_min: i64,
    pub tau_max: i64,
}

pub struct PathContext<'a> {
    pub path: &'a [String],
    pub caches: &'a ScoringCaches<'a>,
}
```

Five built-in components: `EntityRarity`, `EdgeRarity`, `NeighborhoodConcentration`, `TemporalNovelty`, `GnnThreat`. Each lives in `graph\_hunter\_core/src/scoring/components.rs` as a unit struct implementing `ScoreComponent`.

22.2 Composite orchestrator

`CompositeScorer` holds `Vec<Box<dyn ScoreComponent>>` plus matching weights and does a $\sum w_i c_i / \sum w_i$ weighted average. Zero weight disables a component (it's still evaluated but contributes 0).

22.3 Migration

`AnomalyScorer::score\_path\_v2` is a drop-in alternative to the hardcoded `score\_path`. Both coexist for one release; `graph\_hunter\_core/tests/scoring\_snapshot.rs` pins bit-equivalence. A follow-up flips the default and deletes v1.

In the code.

- `graph\_hunter\_core/src/scoring/mod.rs` — trait + `ScoreOutcome`.
- `graph\_hunter\_core/src/scoring/composite.rs` — `CompositeScorer`.
- `graph\_hunter\_core/src/scoring/components.rs` — 5 component impls.

Chapter 23

StringInternerBackend Trait

Note. Stub. See Chapter 4 for the underlying struct and metrics. The trait exists as a seam for a future `GenerationalInterner` or `PartitionedInterner`; no second impl is shipped today, the trait is pre-positioned for when production evidence justifies one.

23.1 Contract

```
pub trait StringInternerBackend {
    fn intern(&mut self, s: &str) -> StrId;
    fn resolve(&self, id: StrId) -> &str;
    fn try_resolve(&self, id: StrId) -> Option<&str>;
    fn get(&self, s: &str) -> Option<StrId>;
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool;
    fn reserve(&mut self, additional: usize);
    fn metrics(&self) -> InternerMetrics;
}

pub struct InternerMetrics {
    pub unique_strings: usize,
    pub approx_bytes:   usize,
}
```

23.2 Trigger conditions

A second backend is worth implementing when any of:

- `metrics().unique\_strings` exceeds $\sim 10^7$ in a live session.
- A streaming session runs 24×7 for more than 72h without compaction.
- A single process serves multiple tenants (MSSP deployment).

The engine exposes the metrics via `api.get\_interner\_metrics()` so an operator can observe the approach to a trigger rather than hit it blind.

In the code.

`graph\_hunter\_core/src/interner.rs` — trait + metrics.
`TODO.md` — trigger criteria.

Chapter 24

IngestAdapter Trait

Note. Stub. The P2-E refactor introduced a higher-level wrapper around `LogSource + LogParser`: new source authors implement `IngestAdapter` in a child crate rather than editing core files.

24.1 Contract

```
#[async_trait]
pub trait IngestAdapter: Send + Sync {
    fn adapter_id(&self) -> &str;
    fn tables(&self) -> &[&'static str];
    async fn stream_triples(
        &self,
        table: &str,
        since: Watermark,
        cancel: CancellationToken,
    ) -> Result<TripleBatchStream, SourceError>;
}

pub struct TripleBatch {
    pub triples: Vec<ParsedTriple>,
    pub watermark: Watermark,
    pub fetched_at: i64,
}
```

24.2 Generic implementation

`LogSourceAdapter` wraps any `LogSource + LogParser` into an `IngestAdapter`. The 4 existing sources (Log Analytics, Defender XDR, Data Lake, AAD export) can be exposed as adapters with zero source-code change; the orchestrator still holds them in concrete form today for backward compatibility, but the seam is open.

24.3 MVCC propagation

`TripleBatch.fetched_at` carries the upstream fetch timestamp, which is exactly what the writer needs to call `CompactRelation::with_ingested_at` on each inserted edge — the third leg of the P3 MVCC story (Chapter 19). Wiring that plumbing end-to-end is a deferred follow-up.

In the code.

`graph_hunter_core/src/ingest_adapter.rs` — trait + `LogSourceAdapter`.

Chapter 25

YAML Catalog Loader

Note. Stub. The P2-C refactor moved the ATT&CK hypothesis catalog from a compile-time `const` array to 29 embedded YAMLs plus an optional user-override directory.

25.1 Precedence

Later sources override earlier ones by catalog id:

1. **Built-in** — 29 files embedded via `include\_str!` from `graph\_hunter\_core/src/catalog/builtin/cat-N`
2. **User directory** — `\$GRAPHHUNTER\_CATALOG\_DIR` if set; else `\$USERPROFILE/.graphhunter/catalog/` (Windows) or `\$HOME/.graphhunter/catalog/` (Unix). Silent when missing.

25.2 Validation

Every loaded entry passes through a validation pass:

- `id` matches `[A-Za-z0-9\-\_]+`.
- `mitre\_id` matches `T\textbackslash d\{4\}(\textbackslash .\textbackslash d\{3\})?`.
- `dsl\_pattern` parses via `parse\_dsl`.
- `k\_simplicity` ≥ 1 .

Malformed user files do not panic or break startup: the loader logs a `CatalogLoadError` and continues. Built-in YAMLs are structurally covered by a golden test (`graph\_hunter\_core/tests/catalog\_golden`) and by the `include\_str!` compile-time check.

25.3 Diagnostics

`api.get\_catalog\_diagnostics()` returns the collected `Vec<CatalogLoadError>`. The transports surface it via `cmd\_get\_catalog\_diagnostics` (Tauri) and `GET /catalog/diagnostics` (HTTP).

In the code.

`graph\_hunter\_core/src/catalog/mod.rs` — public API (`get\_catalog`, `get\_catalog\_snapshot`).
`graph\_hunter\_core/src/catalog/loader.rs` — loading + validation.
`graph\_hunter\_core/src/catalog/builtin/*.yaml` — 29 built-in entries.

Part VI

Transport Layer

Chapter 26

The Canonical API Façade

Note. Stub. `GraphHunterApi` is the single object every transport (Tauri, HTTP, MCP) holds an `Arc` to. All session management, hunt cache, AI conversation, Sentinel streaming, GNN scorer, and GeoIP state lives inside `ApiState`.

26.1 State

```
pub struct GraphHunterApi(Arc<ApiState>);

struct ApiState {
    sessions: SessionStore,
    hunt_cache: Arc<RwLock<Vec<Vec<String>>>>,
    npu_scorer: Arc<RwLock<Option<NpuScorer>>>,
    geoip_provider: Arc<Mutex<Option<MaxMindProvider>>>,
    http_geoip_client: Arc<Mutex<Option<Arc<dyn HttpClient>>>>,
    sentinel_connector: Arc<RwLock<Option<SentinelConnectorHandle>>>,
    ai_config: Arc<RwLock<Option<ProviderConfig>>>,
    emitter: Arc<dyn EventEmitter>,
    // ... 8 more fields
}
```

The façade was introduced by the P1 refactor. Before it, each transport held its own state in a different shape; with the façade in place, Tauri / HTTP / MCP are strictly delegation layers.

26.2 Operations module

Each method on `GraphHunterApi` is defined in `graph\_hunter\_api/src/operations/*.rs`, one file per category (analytics, anomaly, dataset, dsl, enrichment, entity, export, geoip, graph_ops, hunt, ingestion, sentinel, session, sigma, ticket, ai). A method accepts a request DTO, dispatches the session if needed, calls the engine, and returns a response DTO.

26.3 Session handle

`SessionHandle(String)` is an opaque session ID; requests either carry one (explicit target) or omit it (use current session). The façade resolves the handle in one place (`SessionStore::resolve`) so every operation inherits the same precedence logic.

In the code.

`graph\_hunter\_api/src/lib.rs` — `GraphHunterApi` façade, `ApiState`.
`graph\_hunter\_api/src/state/session.rs` — `Session`, `SessionHandle`, `SessionStore`.

Chapter 27

DTOs

Note. Stub. Every operation between a transport and the façade uses a DTO pair (`*Request + *Response`) that lives in `graph\_hunter\_api/src/dto/`. The DTOs are the wire-level contract: if a field is renamed here, every transport’s `serde` output changes identically, and the parity harness (Chapter 29) catches the drift at test time.

27.1 Module layout

One file per operation category, mirroring operations/: `ai.rs`, `analytics.rs`, `anomaly.rs`, `dataset.rs`, `dsl.rs`, `enrichment.rs`, `entity.rs`, `export.rs`, `geoip.rs`, `graph\_ops.rs`, `hunt.rs`, `ingestion.rs`, `sentinel.rs`, `session.rs`, `sigma.rs`, `ticket.rs`.

27.2 Common patterns

- Every request DTO that touches session state embeds `session: Option<SessionHandle>`. The façade resolves `None` to the current session.
- Errors flow via `Result<T, ApiError>` (see `graph\_hunter\_api/src/error.rs`) which every transport maps into its own envelope (HTTP 400/404/500, Tauri `CommandError`, MCP tool error text).
- Enum variants use `\#[serde(tag = "kind")]` for struct variants; newtype variants are serialised as their plain string display (a `serde` limitation we documented in Chapter 29’s error-roundtrip test).

27.3 Shape invariants

The parity harness’s *Phase A* pins every DTO’s key set: a rename silently changes the JSON sent over the wire, and two transports built on the same façade would still disagree if one had cached a stale DTO copy.

In the code.

`graph\_hunter\_api/src/dto/*.rs` — all DTOs.
`graph\_hunter\_api/src/error.rs` — `ApiError`.
`graph\_hunter\_api/tests/parity/dto\_shapes.rs` — Phase A shape pinning.

Chapter 28

Transport Adapters

Note. Stub. Three transports delegate to `GraphHunterApi`: the Tauri desktop app, the axum-based HTTP server, and the TypeScript MCP server. All three are meant to be thin — no logic beyond constructing a request DTO and rendering the response.

28.1 Tauri

Commands live in `app/src-tauri/src/commands/*.rs`. Every command body is ≤ 10 LOC:

```
#[tauri::command]
pub fn cmd_run_hunt(
    api: State<Arc<GraphHunterApi>>,
    hypothesis: Hypothesis,
    time_window: Option<(i64, i64)>,
    max_results: Option<u32>,
) -> Result<RunHuntResponse, CommandError> {
    api.run_hunt(RunHuntRequest {
        session: None,
        hypothesis,
        time_window,
        max_results,
    })
    .map_err(CommandError::from)
}
```

Commands are registered in `app/src-tauri/src/lib.rs` with the `tauri::Builder::invoke_handler` macro.

28.2 HTTP

Handlers live in `app/src-tauri/src/http/_api.rs` and share the same `Arc<GraphHunterApi>` via axum's `State`. The HTTP port is chosen at startup (`util::resolve_api_port`) and written to the Tauri app's config directory so the MCP server and CLI can find it.

Every handler is also ≤ 10 LOC:

```
async fn handler_run_hunt(
    State(api): State<Arc<GraphHunterApi>>,
    Json(body): Json<RunHuntRequest>,
) -> Response {
    match api.run_hunt(body) {
        Ok(v) => ok_json(v),
        Err(e) => error_json(e),
    }
}
```



```
}

```

28.3 MCP

`graph-hunter-mcp/src/index.ts` exposes ~30 tools over the MCP protocol. Each tool handler HTTPs-calls the local engine via `http://127.0.0.1:<port>`; it does not link the Rust engine at all. This is the only transport that doesn't hold an `Arc<GraphHunterApi>` directly — because it runs in a different process (Node.js) — yet it sees the exact same semantics because the HTTP layer is a thin wrapper around the façade.

28.4 EventEmitter

Progress events (session load, ingest, Sentinel streaming) flow through an `Arc<dyn EventEmitter>` stored on `ApiState`. `TauriEventEmitter` bridges to `AppHandle::emit`; `NoopEmitter` is used by HTTP, MCP, CLI, and tests.

In the code.

- `app/src-tauri/src/commands/` — Tauri command modules.
- `app/src-tauri/src/http/_api.rs` — HTTP handlers.
- `graph-hunter-mcp/src/index.ts` — MCP tools.
- `graph\_hunter\_api/src/events.rs` (or similar) — `EventEmitter` trait.

Chapter 29

Parity Harness

Note. Stub. The harness lives in `graph\_hunter\_api/tests/parity*.rs`. Two phases ship today (A: DTO shape; B: scenario); a third phase covering the HTTP + Tauri runtime is deferred.

29.1 Phase A — DTO shape

For every request/response DTO pair, the harness serialises a hand-crafted value to JSON and asserts the object's top-level keys match an expected set exactly. A missed field or a rename yields a clear failure at test time.

The helper:

```
fn assert_keys<T: Serialize>(value: &T, expected: &[&str]) {
    let v = serde_json::to_value(value).unwrap();
    let actual: BTreeSet<_> = v.as_object().unwrap()
        .keys().map(|s| s.as_str()).collect();
    let want: BTreeSet<_> = expected.iter().copied().collect();
    assert_eq!(actual, want);
}
```

29.2 Phase B — Scenario

End-to-end scenarios exercise ordered operation sequences against the façade directly (not through a transport). The harness builds a fixture session with a small seeded graph (Alice + Bob authenticating to web-01, web-01 executing `cmd.exe`), runs a sequence of API calls, and asserts observable state at each step.

Fixtures:

- `fresh\_api\_with\_session()` — empty fresh API plus a single session.
- `fresh\_api\_with\_seeded\_session()` — the seeded 4-entity / 3-edge graph.
- `tag`, `note`, `parse` — convenience helpers so scenarios read at the operation level.

Sample assertions: `run\_hunt\_populates\_cache\_and\_pages\_back` (a full run → page-back round-trip matches), `diff\_hunts\_rejects\_inverted\_window` (the invariant from Chapter 15), `export\_iocs\_sentinel\_watchlist\_csv\_contains\_tagged\_ips\_only` (the export format).

29.3 Deferred Phase C

A full HTTP + Tauri runtime harness (spin up axum in the test, wire a Tauri IPC mock) is flagged as follow-up. Phase A + B together catch the shape and semantic drifts that would have broken parity between transports; Phase C would add wire-level round-trips.

In the code.

`graph\_hunter\_api/tests/parity.rs` — harness entry.
`graph\_hunter\_api/tests/parity/fixture.rs` — fixture builders.
`graph\_hunter\_api/tests/parity/dto\_shapes.rs` — Phase A.
`graph\_hunter\_api/tests/parity/scenarios.rs` — Phase B.

Chapter 30

Session Persistence

Note. Stub. Sessions are serialised to JSON files under the OS's app-data directory. The format is currently `schema_version = 1`; a v2 bump for the P3 MVCC migration is planned (Chapter 19).

30.1 On-disk layout

- Windows: `\\%APPDATA%\\textbackslash GraphHunter\\textbackslash sessions\\textbackslash <id>.json`
- Unix: `\\$HOME/.local/share/GraphHunter/sessions/<id>.json`

`<id>` is validated against `[A-Za-z0-9\\_\\-]\\{1,128\\}` to prevent path traversal. The path is constructed via `PathBuf::join`, never string concatenation.

30.2 SessionFile shape

```
pub struct SessionFile {  
    pub id: String,  
    pub name: String,  
    pub created_at: i64,  
    pub entities: Vec<Entity>,  
    pub relations: Vec<Relation>,  
    pub path_node_ids: Vec<String>,  
    pub notes: Vec<Note>,  
    pub datasets: Vec<DatasetInfo>,  
    pub tags: Vec<EntityTag>,  
    // NO ai_conversation -- chat is deliberately ephemeral.  
}
```

30.3 Load progress

Loading is async. The `EventEmitter` receives `SESSION\\_LOAD\\_PROGRESS` events at each stage: parsing the file, inserting entities, inserting relations, rebuilding indexes (scorer finalize), done. The UI renders a progress bar; HTTP / MCP simply ignore the events (`NoopEmitter`).

30.4 v2 migration (deferred)

When the P3 MVCC writer change lands, legacy edges (loaded from v1 files) must be backfilled with a synthetic `ingested\\_at`. The plan is to use the edge's `timestamp` as the backfill value: it's

not exactly right (the edge was ingested at some later moment), but it's a defined, deterministic choice that lets v2 snapshot queries produce sensible results against v1 data. The UI gets a "approximate snapshot" banner so the analyst knows.

In the code.

graph_hunter_api/src/state/persistence.rs — load / save.
graph_hunter_api/src/state/session_types.rs — SessionFile, Note, EntityTag.

Appendix A

Complexity Summary

One-page reference of the dominant operations across the engine, with time and space complexity. Chapter references point at the full derivation.

Operation	Time	Space	Ch.	File pointer
INTERN	$O(1)$ am.	$O(s)$ miss	4	interner.rs:60
RESOLVE	$O(1)$	—	4	interner.rs:74
ADD-ENTITY	$O(1)$ am.	$O(1)$	5	graph.rs:211
ADD-RELATION	$O(1)$ am.	$O(32))$	5	graph.rs:227
SORT-EDGES-BY-Ts	$O(m \log \bar{d})$	$O(1)$	5	graph.rs:437
SPILL-APPEND	$O(1)$ am.	$O(L)$	6	spill.rs
K-WAY-MERGE	$O(n \log k)$	$O(k)$	6	spill.rs:327
METADATA-APPEND	$O(K)$	$O(K)$	7	metadata_store.rs
PARSE-DSL	$O(src)$	$O(src)$	9	dsl.rs:47
DFS-MATCH-ITERATIVE	$O(n_s \bar{b}^\ell \ell)$	$O(W \ell k)$	10	graph.rs:1121
MERGE-INTO-TOPK	$O((a + b) \log K)$	$O(K)$	10	graph.rs:782
INTERSECT-SORTED-U32	$O(S L /8)$	$O(1)$	11	simd_rust.rs
ANOMALY-FINALIZE	$O(V + E)$	$O(V)$	12	anomaly.rs:148
COMPUTE-NC	$O(d(v))$	$O(\mathcal{T}_V)$	12	anomaly.rs:191
SCORE-PATH	$O(\ell)$	$O(1)$	12	anomaly.rs:241
COMPUTE-BETWEENNESS	$O(S(V + E))$	$O(V + E)$	13	analytics.rs:641
TEMPORAL-PAGERANK	$O(I(V + E))$	$O(V)$	13	analytics.rs:746
GET-RELATION-SCHEMA	$O(E + T \log T)$	$O(T)$	13	analytics.rs:1289
SCORE-AND-PAGINATE	$O(P\ell + U \log U)$	$O(U)$	14	analytics.rs:1096
DIFF-HUNTS	$O(2T_{\text{match}} + P)$	$O(P)$	15	analytics.rs:1342
ENCODE-INGESTED-AT	$O(1)$	$O(1)$	3	relation.rs:151
CHECK-STEP-COMPAT	$O(\Sigma)$	$O(1)$	8	catalog/mod.rs:105

Table A.1: Complexity summary. n = vertices, m = edges, $\bar{d}$ = avg degree, $d(v)$ = degree of v , ℓ = hypothesis step count, k = k -simplicity, n_s = number of candidate start vertices, $\bar{b}$ = avg candidate branching factor, W = rayon worker count, K = top- K cap, L = spill limit, S = sampled sources, I = iteration count, P = paths pre-dedup, U = paths post-dedup, T = unique (src, rel, dst) triples.

Appendix B

Catalog Reference

The 29 built-in ATT&CK hypothesis patterns. Each entry lives as a separate YAML in `graph\_hunter\_core/src` and is validated + cached on first access (Chapter 25).

ID	MITRE	Pattern	k
cat-001	T1078	User → Host → Process	1
cat-002	T1059.001	User → Process → Process → File	1
cat-003	T1021.001	IP → Host → User → Process	1
cat-004	T1003	User → Process → File	1
cat-005	T1071	Process → Domain → IP	1
cat-006	T1569.002	User → Process → Service	1
cat-007	T1053.005	Process → File → Process	1
cat-008	T1055	Process → Process → File	1
cat-009	T1048.003	User → Process → Domain	1
cat-010	T1204.002	Process → File → Process	1
cat-011	T1547.001	Process → Registry → Process	1
cat-012	T1021	User → Host → Process → File	1
cat-013	T1071	Process → Domain → IP → Process (cycle)	2
cat-014	T1547.001	Process → Registry → Process (cycle)	2
cat-015	T1021	Host → User → Process → Host (cycle)	2
cat-016	T1059	Process → Process → Process (cycle)	2
cat-017	T1110.003	User {status=Failure} → IP HAVING count(distinct IP) ≥ 3 WITHIN 24h	1
cat-018	T1087.004	User {app~PowerShell} → IP	1
cat-019	T1078.004	User {risk_tag=anonymizing} → IP	1
cat-020	T1110.003	User {status=Success} → IP HAVING count(distinct IP) ≥ 2	1
cat-021	T1078.004	User → IP HAVING count(distinct IP) ≥ 2 WITHIN 6h	1
cat-022	T1528	User → OAuthApp	1
cat-023	T1098.003	User → ServicePrincipal	1
cat-024	T1098.003	User → Role → User	1
cat-025	T1564.008	User → MailboxRule → Domain	1
cat-026	T1218	Process → Process {name~certutil}	1
cat-027	T1014	Process → File {path~.sys}	1
cat-028	T1562.001	Process → Service {name~Sense}	1
cat-029	T1053.005	User → Host → Task → Process	1

Table B.1: Built-in catalog. Arrows elide relation types for compactness; see the YAML for exact patterns.

Entries 017–021 exercise the metadata-predicate grammar; 022–025 are the cloud-identity patterns requiring **Consent**, **Create**, **Assign**, **Forward** relation types; 026–028 are the endpoint-modernisation patterns using substring matchers on destination-vertex metadata.

Appendix C

Benchmarks

Two Criterion benchmark suites live in `graph\_hunter\_core/benches/`:

- `hunt\_latency.rs` — end-to-end `search\_temporal\_pattern` timing on a synthetic graph. Used as the " $\leq 5\%$ regression" gate by the P1/P2 refactor plans.
- `dedup\_throughput.rs` — `SpillableEdgeStore` ingest timing. Validates the k-way merge's $O(k)$ -RAM claim on a 2M-record sample (0.23 MB delta vs the ~ 92 MB of the old Vec-based approach).

C.1 Running

```
cd graph_hunter_core
cargo bench --bench hunt_latency
cargo bench --bench dedup_throughput
```

Criterion's default output lands under `graph\_hunter\_core/target/criterion/` as HTML reports.

C.2 Current baseline (illustrative)

Benchmark	Result	Notes
<code>hunt_latency</code> 2-step, 10K edges	$\sim 200\ \mu\text{s}$	scalar Rust
<code>hunt_latency</code> 2-step, 10K edges, SIMD	$\sim 70\ \mu\text{s}$	libgraphmatch
<code>dedup_throughput</code> 2M records	$\sim 1.5\ \text{s}$	spill mode

Table C.1: Illustrative benchmark numbers on a laptop-class CPU (AMD Ryzen 7 5800H). Re-run locally for up-to-date numbers; actual values drift per hardware and build flags.

C.3 Three-way SIMD comparison (planned)

The P2-D follow-up adds a third column to the hunt-latency table: pure-Rust `simd\_rust`. The comparison should show (a) scalar, (b) libgraphmatch FFI, (c) Rust intrinsics. The expectation is that Rust intrinsics reach ~ 70 – 80% of libgraphmatch's peak, at which point the operational benefit of dropping the C++ toolchain dominates.

Appendix D

Source Index

file:line pointers for every algorithm named in the document.

D.1 graph_hunter_core/src/

- anomaly.rs:148 — ANOMALY-FINALIZE
- anomaly.rs:191 — COMPUTE-NC
- anomaly.rs:241 — SCORE-PATH (v1)
- anomaly.rs:351 — node_anomaly_estimate
- analytics.rs:289 — build_narrative
- analytics.rs:641 — COMPUTE-BETWEENNESS
- analytics.rs:746 — TEMPORAL-PAGERANK
- analytics.rs:1096 — SCORE-AND-PAGINATE
- analytics.rs:1289 — get_relation_schema
- analytics.rs:1342 — DIFF-HUNTS
- backend/mod.rs — GraphRead, GraphWrite
- backend/in_memory.rs — trait impls
- catalog/mod.rs:105 — CHECK-STEP-COMPAT
- catalog/loader.rs — YAML load + validation
- dsl.rs:47 — PARSE-DSL
- graph.rs:67 — InMemoryGraph struct
- graph.rs:211 — ADD-ENTITY
- graph.rs:227 — ADD-RELATION
- graph.rs:437 — SORT-EDGES-BY-TIMESTAMP
- graph.rs:461 — SEARCH-TEMPORAL-PATTERN
- graph.rs:689 — SEARCH-TEMPORAL-PATTERN-SIMD
- graph.rs:782 — SEARCH-TEMPORAL-PATTERN-SMART
- graph.rs:932 — DFS-MATCH-SMART
- graph.rs:1121 — DFS-MATCH-ITERATIVE
- hypothesis.rs — Hypothesis, HypothesisStep, Predicate
- ingest/batch.rs — ParsedBatch

- `ingest/channels.rs` — bounded channels
- `ingest/enrichment.rs` — enrichment preemption
- `ingest/overflow.rs` — queue persistence
- `ingest/parser\_task.rs` — parser task
- `ingest/source\_poller.rs` — polling loop
- `ingest/writer.rs` — `GraphWriter`
- `ingest\_adapter.rs` — `IngestAdapter`, `LogSourceAdapter`
- `interner.rs:60` — `INTERN`
- `interner.rs:74` — `RESOLVE`
- `metadata\_store.rs` — metadata append / read
- `relation.rs:83` — `CompactRelation` struct
- `relation.rs:151` — `ENCODE-INGESTED-AT`
- `scoring/mod.rs` — `ScoreComponent` trait
- `scoring/composite.rs` — `CompositeScorer`
- `scoring/components.rs` — 5 components
- `simd\_matcher.rs` — `libgraphmatch` FFI
- `simd\_rust.rs` — `INTERSECT-SORTED-U32`
- `spill.rs:327` — `K-WAY-MERGE` finalize
- `spill.rs:524` — `GET-EDGES`
- `sources/mod.rs:275` — `LogSource` trait

D.2 `graph_hunter_api/src/`

- `lib.rs` — `GraphHunterApi` façade, `ApiState`
- `dto/*.rs` — request/response DTOs
- `operations/*.rs` — per-category operations
- `state/session.rs` — `Session`, `SessionStore`, `SessionHandle`
- `state/persistence.rs` — session load/save
- `state/session\_types.rs` — `SessionFile`, `Note`, `EntityTag`
- `tests/parity.rs` — harness entry
- `tests/parity/dto\_shapes.rs` — Phase A
- `tests/parity/scenarios.rs` — Phase B
- `tests/parity/fixture.rs` — fixture builders

D.3 `app/src-tauri/`

- `src/commands/` — Tauri command modules
- `src/http\_api.rs` — HTTP handlers
- `src/lib.rs` — Tauri Builder registration
- `src/tauri\_emitter.rs` — `TauriEventEmitter`

D.4 graph-hunter-mcp/

- `src/index.ts` — all MCP tool handlers

D.5 libgraphmatch/

- `include/graphmatch/csr\_graph.hpp` — `CSRTemporalGraph` (`add\_node`, `add\_edge`, `finalize`)
- `src/simd\_intersect.cpp` — AVX2 sorted-set intersection
- `src/matcher.cpp` — SIMD pattern matcher entry
- `tests/test\_csr\_graph.cpp` — C++ boundary tests (including the P2-D `add\_edge` boundary suite)